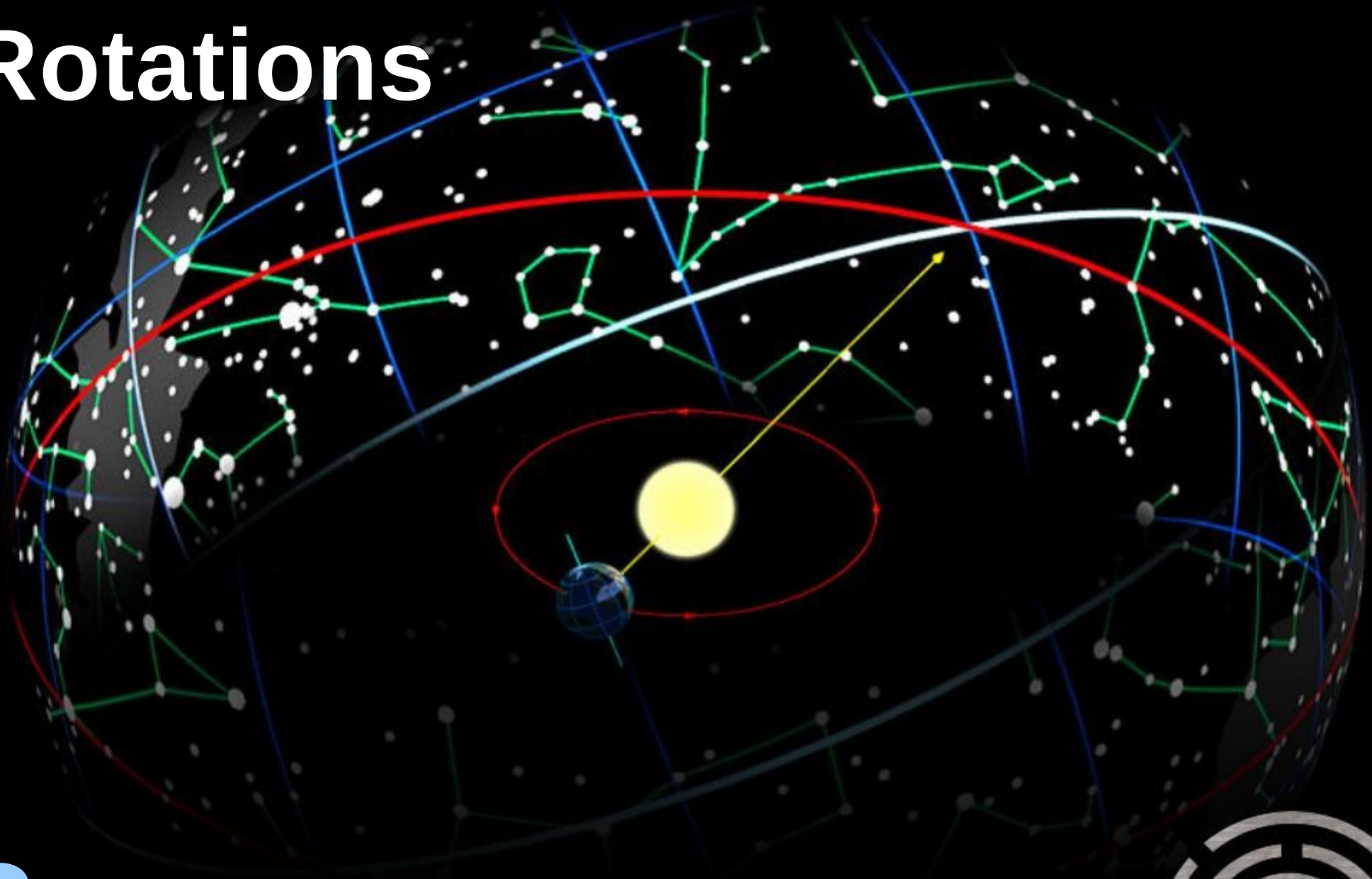
Rotations



Basic Concept 3D Rotation

Every 3D-Rotation can be described as a 2D (on a plane) rotation

Rotation on a 2D plane == Rotation araound a 3D vector (e.g. Quaternion)

Or 3 consecutive rotations, Z, Y, X

... But

Basic Concept 3D Rotation

3 consecutive rotations, Z, Y, X
Do not simply add,
the axes also rotate!
Rotations are non permutable

Microrotations: 3 consecutive rotations (around Z, around Y, around X, e.g. RPY)
→ produce a complete 3D rotation
in very small steps, so the changes of axis do not bother that much
Therefore very high frequency when reading IMU (100 .. 1000/Sec)

This way, every 3D rotation can be (almost perfectly) calculated (executed)

Microrotations -> Attitude

dx/dt -> Δx -> x

Determination of attitude/orientation by IMU:

1. Calibration and definition of initial attitude (airplane ~ 10 minutes)

Loop (100 Hz... 1KHz) {// for infinitesimal,
// The smaller the steps, the smaller the error.

2. read gyroscope: output 3D **rotation rates** $\underline{\omega}$

3. $\underline{\Delta\varphi} = \underline{\omega} \cdot T$ (T = time-Step)

For all X,Y,Z:
Delta Angle =
Rotation Speed *
Time Step

4. $\underline{\Delta\varphi}$ -> Rotor (Matrix, Quaternion, Rotation Axis+Angle, etc.)

5. Attitude = Rotor * Alte_Attitude // Beware with "*" See def.
// matrix * matrix or Q*Q

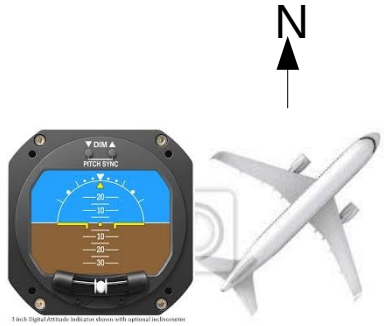
}

Microrotations -> Attitude

dx/dt -> Δx -> x

Determination of attitude/Orientation by an IMU:

1. Calibration and definition of initial attitude // (airplane ~ 10 minutes)
   Offset and Drift
   Attitude = e.g. ((100) (010) (001))



TIME_LOOP(100 Hz... 1KHz) {

2. read gyroscope: returns 3D **rotation rates** $\underline{\omega}$
3. $\underline{\Delta\varphi} = \underline{\omega} \cdot T$ (T = time-Step)
4. $\underline{\Delta\varphi}$ -> Rotor // (Matrix, Quaternion, ratation axis+angle,etc)

```
        computer RotorTotal
    alternatives
        RotorTotal = Rotor(x)*Rotor(y)*Rotor(z) // * is defined for Matrix and Quat.
        Attitude = RotorTotal * Attitude        // We add together Rotations by * ?
    or                                          //  L = (X*Y*Z*)*L
        Attitude = Rotor(z) * Attitude
        Attitude = Rotor(y) * Attitude
        Attitude = Rotor(x) * Attitude
    or
        Attitude = (Rotor(X)*Rotor(Y)*Rotor(Z))*Attitude
}
```

What is better?

What is a Rotor?

Rotation matrix

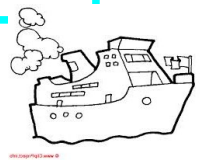
-> 3 axes (x,z,y)
(e.g. walking round the corner)

Quaternions (rotation vector, rotation angle)

Best for calculations
-> 1x Rotation axis
(direct path)

Euler-Angle (incl. RPY)

Only for output (artif. horiz)
and for measurements
(definitely **not** for calculations)



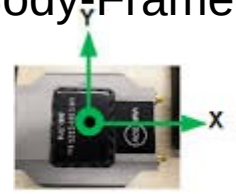
(RPY = Roll, Pitch, Yaw)

Mounting Arrangement of IMU

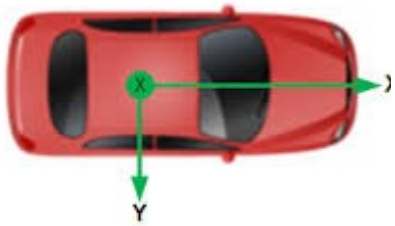
This is the current attitude of the IMU
-> We consider the new attitude of the IMU

Attitude = RotorTotal * Attitude

And if Mounting Arrangement of IMU != Body-Frame?

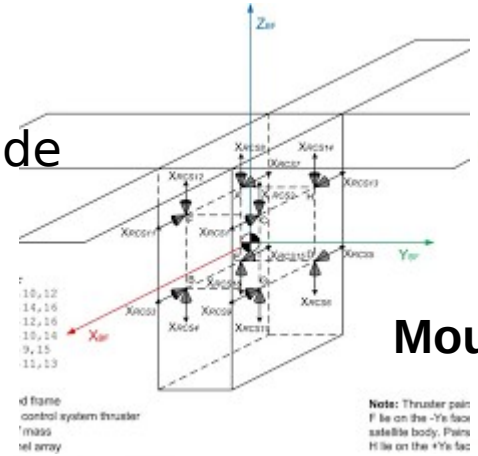


The IMU frame



The body frame

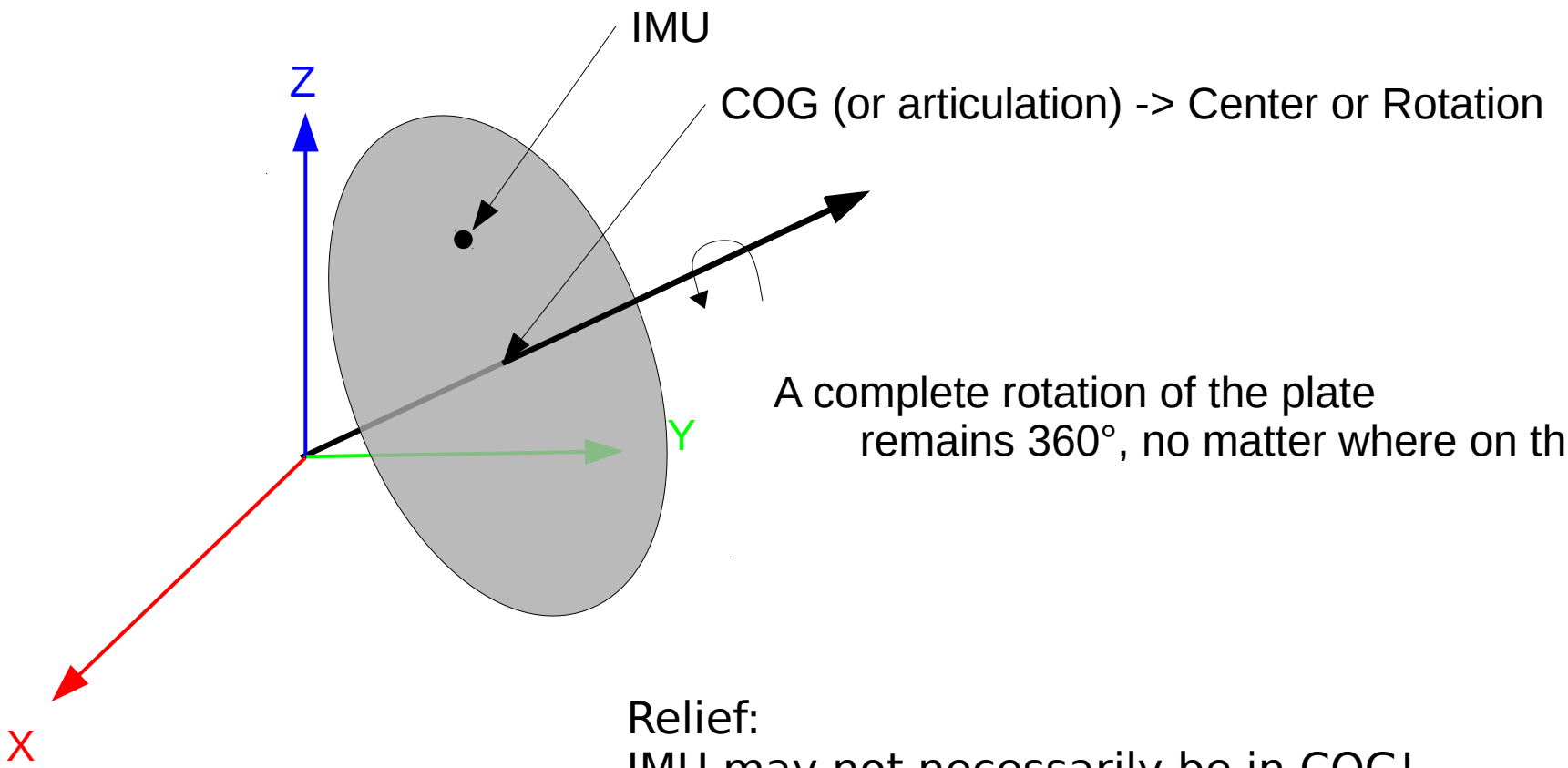
-> a Rotor-“Mounting Arrangement”
IMU_Attitude = RotorTotal * IMU_Attitude
Attitude = Mounting Arrangement* IMU_Attitude
// or the other way round?



Mounting frames

This was angular position relative to what?

Must IMU Be in Center of Gravity / Center of Rotation?



A complete rotation of the plate remains 360°, no matter where on the plate

Relief:
IMU may not necessarily be in COG!
Same is true for wheels

Yet this was inertial!

Okay, this was inertial attitude
However **I** want it relative to **xx**

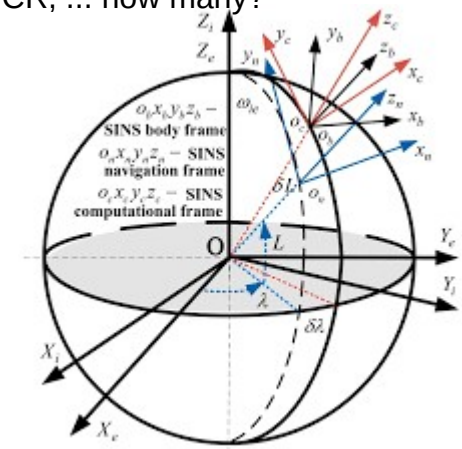
Rot (**I → xx**)= (e.g. Dir. Cos. Matrix)..... // all rights and lefts were correct?

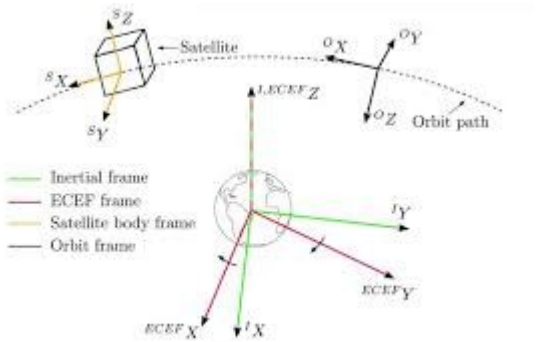
Attitude_**xx** = Attitude * Rot (**I→ xx**) // or the other way round?

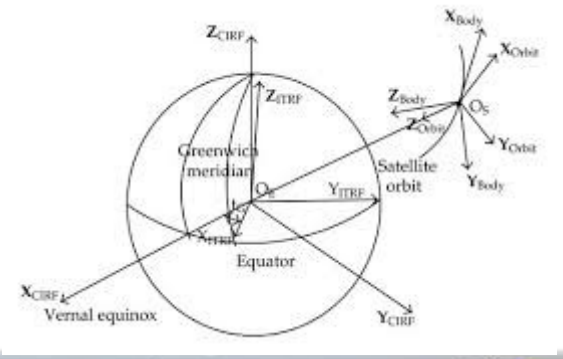
and than from xx to yy and than to vv ....
Rot (xx → yy), Rot(yy → vv) and multiply // all rights and lefts were correct?

(How often we were not confident, whether right or left? n % 2 = 0 hence ok or?

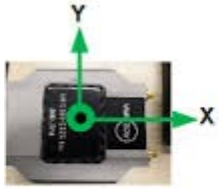
xx,yy,zz z.B. Body-frame, Nav Frame, ECEF, ECI, Orbit:
LHLV, ICR, ... how many?



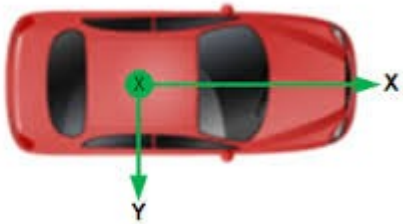




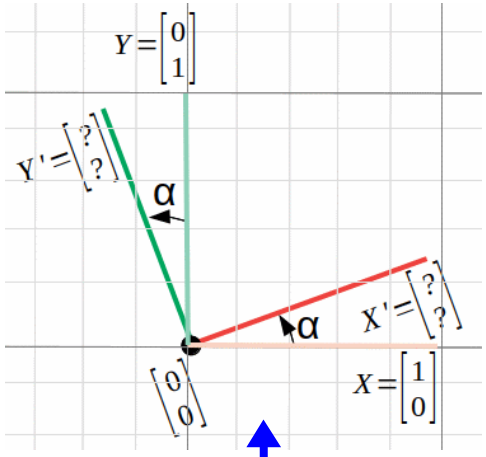
Rotor for Mounting Arrangement / Moving Frames

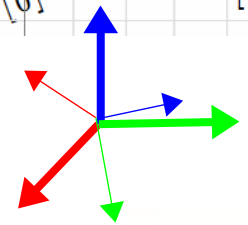


The IMU frame



The body frame

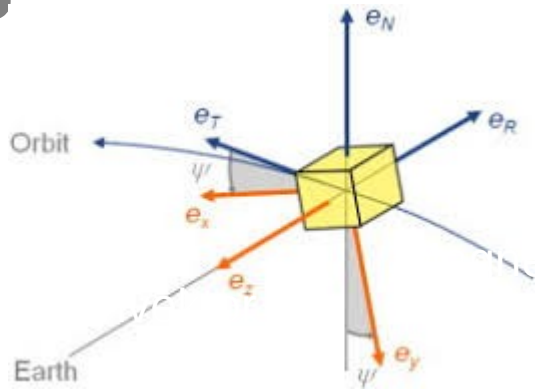




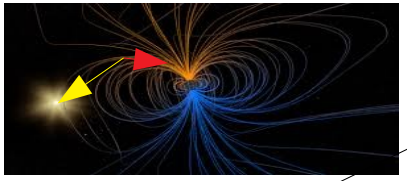
Matrix with (X Y Z)
unit vectors
of IMU
engineering drawing (grads) +
Measurements (10^-4 grad)
**

See following slides

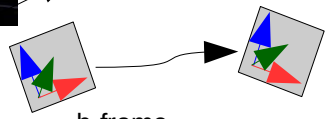
** Measure: e.g. Laserpointer
relative to structure, relative to star sensor
+/- 0,01° in 400 km on the ground 100 m



Direction Cosinus Matrix
with 2 vectors -> 3 unit vectors



X = S
Z = SxM
Y = ZxX



b-frame sm-framek

Alternatives:
DCM or 2 vectors

$$\begin{pmatrix} a_x \bullet b_x & a_x \bullet b_y & a_x \bullet b_z \\ a_y \bullet b_x & a_y \bullet b_y & a_y \bullet b_z \\ a_z \bullet b_x & a_z \bullet b_y & a_z \bullet b_z \end{pmatrix}$$

Relative to what?

And then from “sun-magnet-Frame” to ECI and than ECEF
off the navigation module!

Microrotations -> Attitude

dx/dt -> Δx -> x

Determination of attitude/orientation by IMU:

1. Calibration and definition of initial attitude (airplane ~ 10 minutes)

Loop (100 Hz... 1KHz) {// for infinitesimal,
// The smaller the steps, the smaller the error.

2. read gyroscope: output 3D **rotation rates** $\underline{\omega}$

3. $\underline{\Delta\varphi} = \underline{\omega} \cdot T$ (T = time-Step)

For all X,Y,Z:
Delta Angle =
Rotation Speed *
Time Step

4. $\underline{\Delta\varphi}$ -> Rotor (Matrix, Quaternion, Rotation Axis+Angle, etc.)

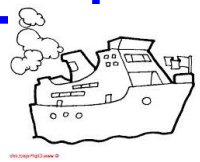
5. Attitude = Rotor * Alte_Attitude // Beware with "*" See def.
// matrix * matrix or Q*Q

}

Calculation of Rotations

Everything in Cartesian coordinate
Polar coordinates would be easier!

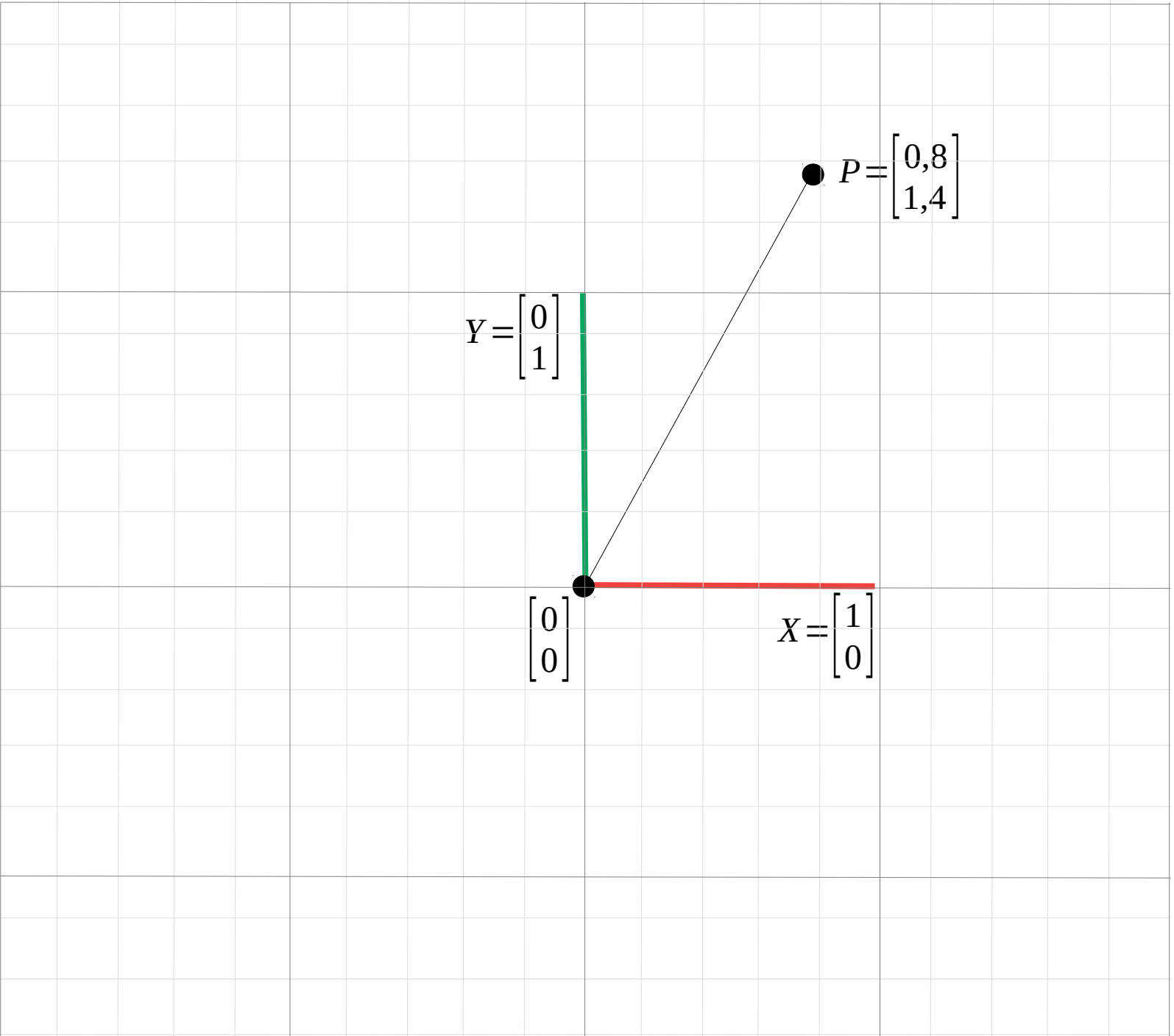
Rotation matrix	-> 3 axes (x,z,y) (e.g. walking round the corner)
Quaternions (rotation vector, rotation angle)	Best for calculations -> 1x Rotation axis (direct path)
Euler-Angle (incl. RPY)	For output (artif. horiz). For measurements (definitely **not** for calculations)



(RPY = Roll, Pitch, Yaw)

Film play-3D-rotation-matrix-euler-vector

With Matrices



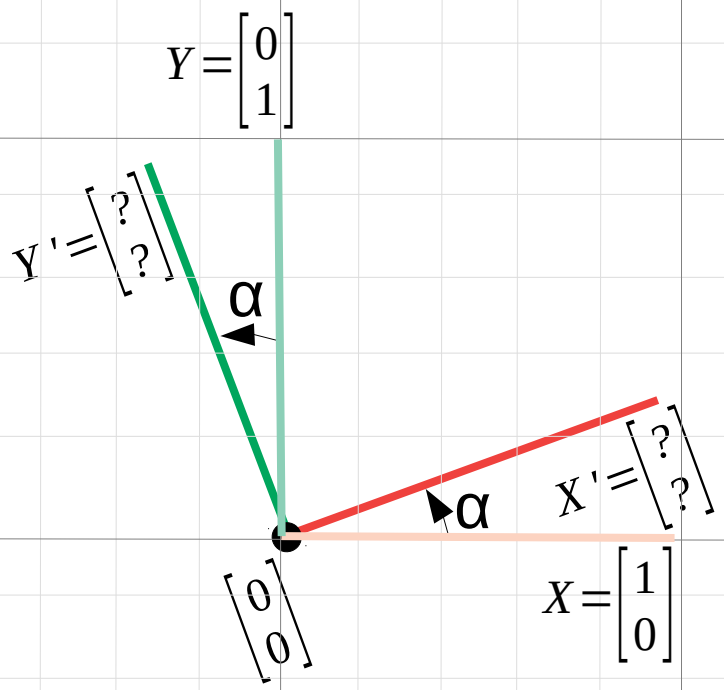
$$P=\begin{bmatrix}0{,}8\\1{,}4\end{bmatrix}$$

$$P=0{,}8*X+1{,}4*Y$$

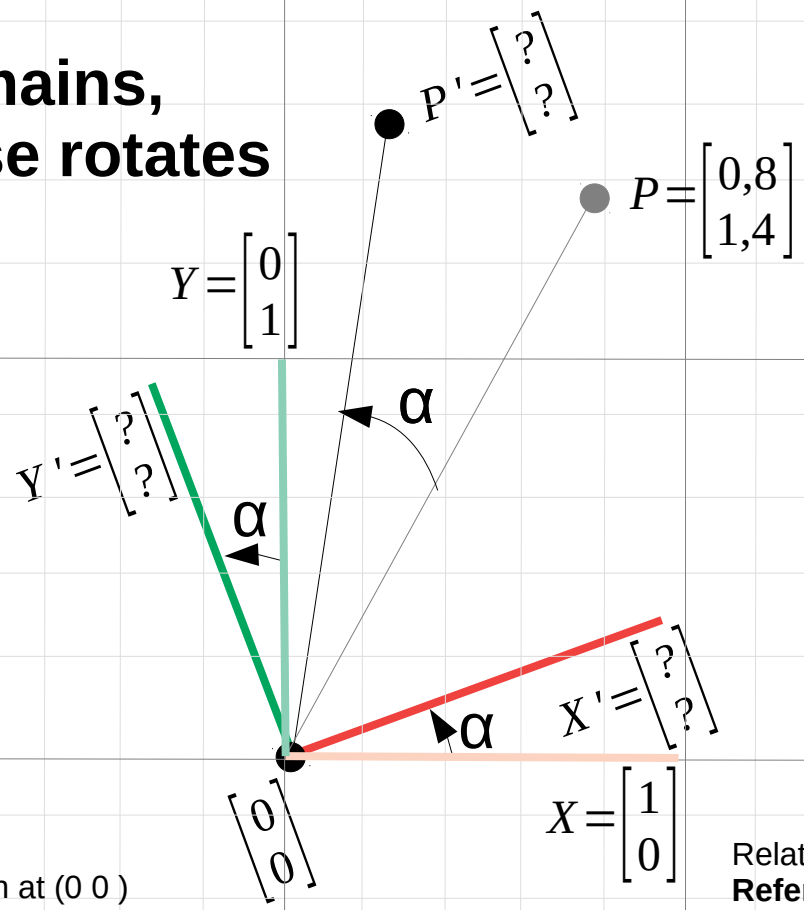
$$P=0{,}8*\begin{bmatrix}1\\0\end{bmatrix}+1{,}4*\begin{bmatrix}0\\1\end{bmatrix}$$

$$P=\begin{bmatrix}0{,}8\\0\end{bmatrix}+\begin{bmatrix}0\\1{,}4\end{bmatrix}=\begin{bmatrix}0{,}8\\1{,}4\end{bmatrix}$$

Big surprise!..
it‘s the same!
Why the big show?


$Y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
$Y' = \begin{bmatrix} ? \\ ? \end{bmatrix}$
α
α
$X' = \begin{bmatrix} ? \\ ? \end{bmatrix}$
$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$
$X = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

Reference remains, everything else rotates (by α)



$$P' = \begin{bmatrix} ? \\ ? \end{bmatrix}$$

$$P = 0{,}8 * X' + 1{,}4 * Y'$$

$$P = 0{,}8 * \begin{bmatrix} ? \\ ? \end{bmatrix} + 1{,}4 * \begin{bmatrix} ? \\ ? \end{bmatrix}$$

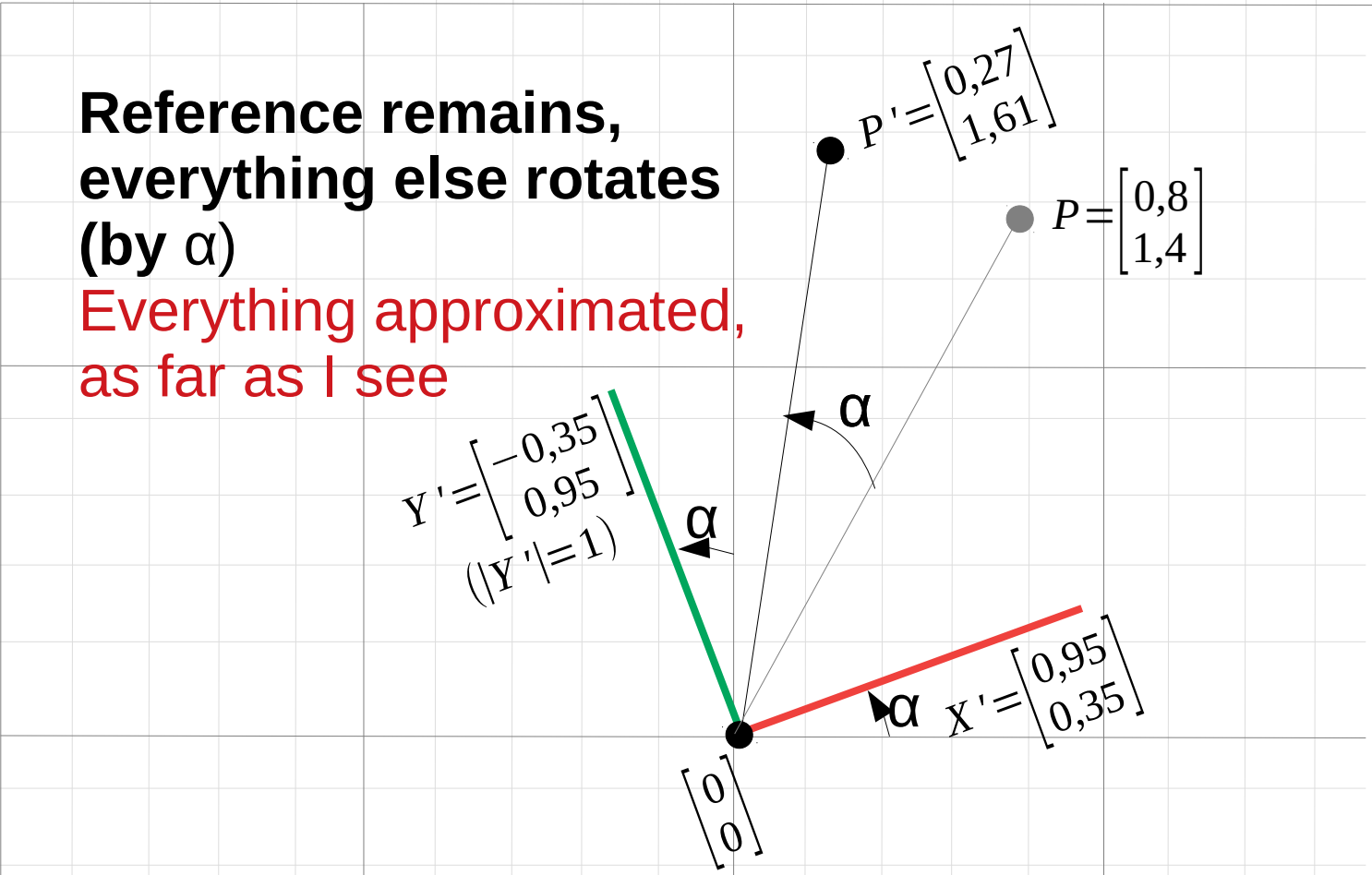
$$P = \begin{bmatrix} 0{,}8 * ? \\ 0{,}8 * ? \end{bmatrix} + \begin{bmatrix} 1{,}4 * ? \\ 1{,}4 * ? \end{bmatrix} = \begin{bmatrix} ? \\ ? \end{bmatrix}$$

We leave origin at (0 0)
-> rotation axis
(CoG or articulation)
IMPORTANT!

Relative to what?
Reference coordinate system:
Earth
ECI
Inertial
My initial attitude

Reference remains, everything else rotates (by α)

Everything approximated, as far as I see



$$P' = \begin{bmatrix} ? \\ ? \end{bmatrix}$$

$$P' = 0{,}8 * X' + 1{,}4 * Y'$$

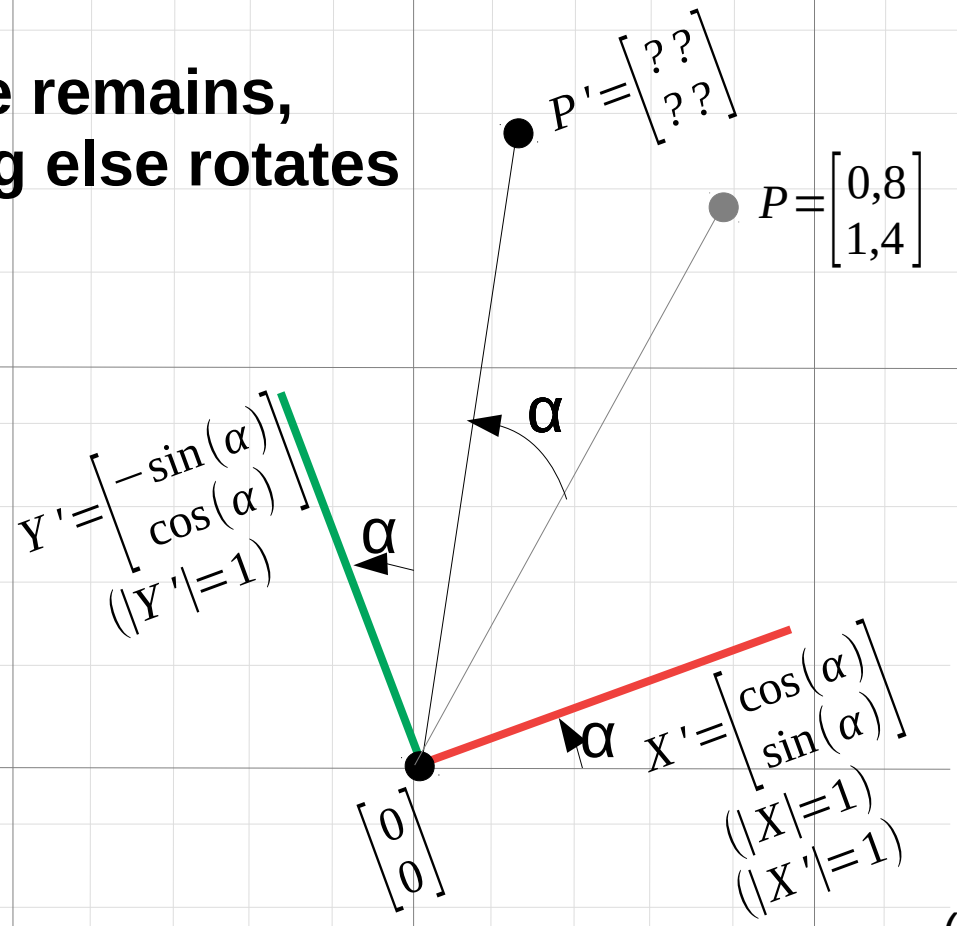
$$P' = 0{,}8 * \begin{bmatrix} 0{,}95 \\ 0{,}35 \end{bmatrix} + 1{,}4 * \begin{bmatrix} 0{,}35 \\ 0{,}95 \end{bmatrix}$$

$$P' = \begin{bmatrix} 0{,}8 * 0{,}95 \\ 0{,}8 * 0{,}35 \end{bmatrix} + \begin{bmatrix} 1{,}4 * -0{,}35 \\ 1{,}4 * \ \ 0{,}95 \end{bmatrix}$$

$$P' = \begin{bmatrix} 0{,}8 * 0{,}95 \ + \ 1{,}4 * -035 \\ 0{,}8 * 0{,}35 \ + \ 1{,}4 * \ \ 0{,}95 \end{bmatrix}$$

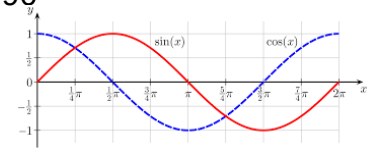
$$P' = \begin{bmatrix} 0{,}27 \\ 1{,}61 \end{bmatrix}$$

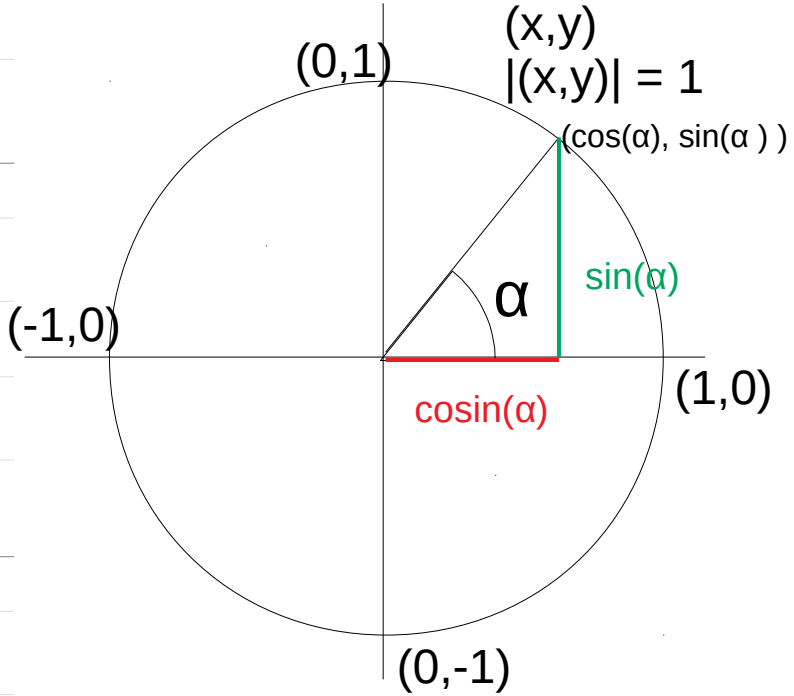
Reference remains, everything else rotates (by α)



Look textbook:
From X to Y = 90°

cos(α + 90) = -sin(α)
cos(90 - α) = sin(α)
sin(90 + α) = cos(α)
...
of course: sin and cos are shifted by 90°



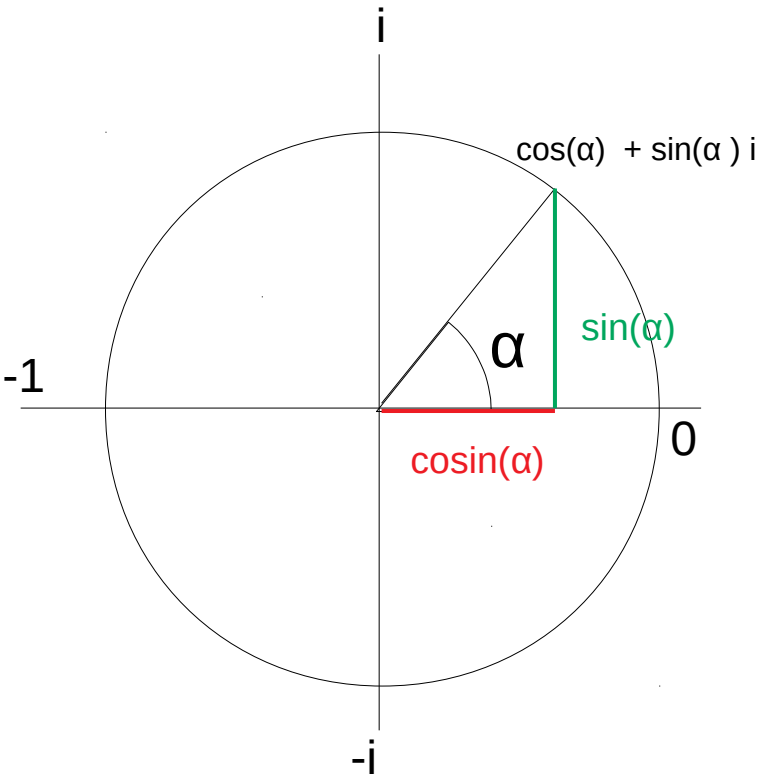


No longer approximately
Now calculate!

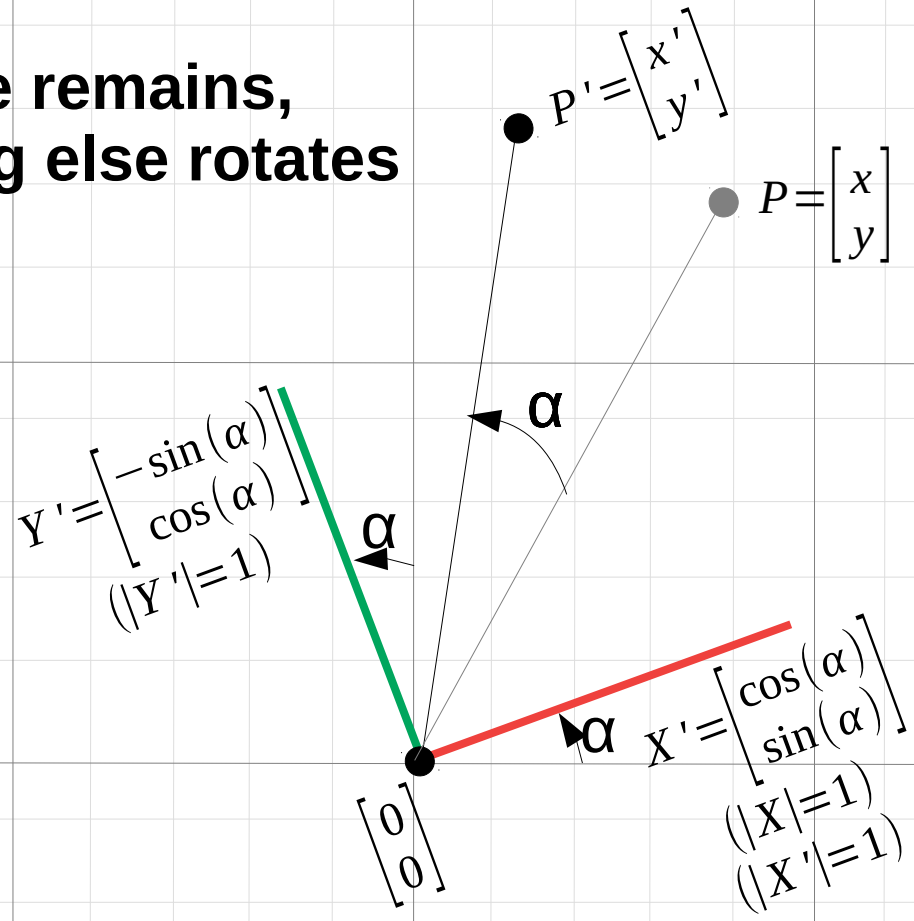
Just to remember

We will need it for quaternions

With complex numbers...
It is easier, to add angles
and it is a preparation
for quaternions.



Reference remains, everything else rotates (by α)



No longer approximately
Now calculate!

P' relativ to body frame
is still the same like P
but relative to reference frame:

$$P' = x * X' + y * Y'$$

$$P' = x * \begin{bmatrix} \cos(\alpha) \\ \sin(\alpha) \end{bmatrix} + y * \begin{bmatrix} -\sin(\alpha) \\ \cos(\alpha) \end{bmatrix}$$

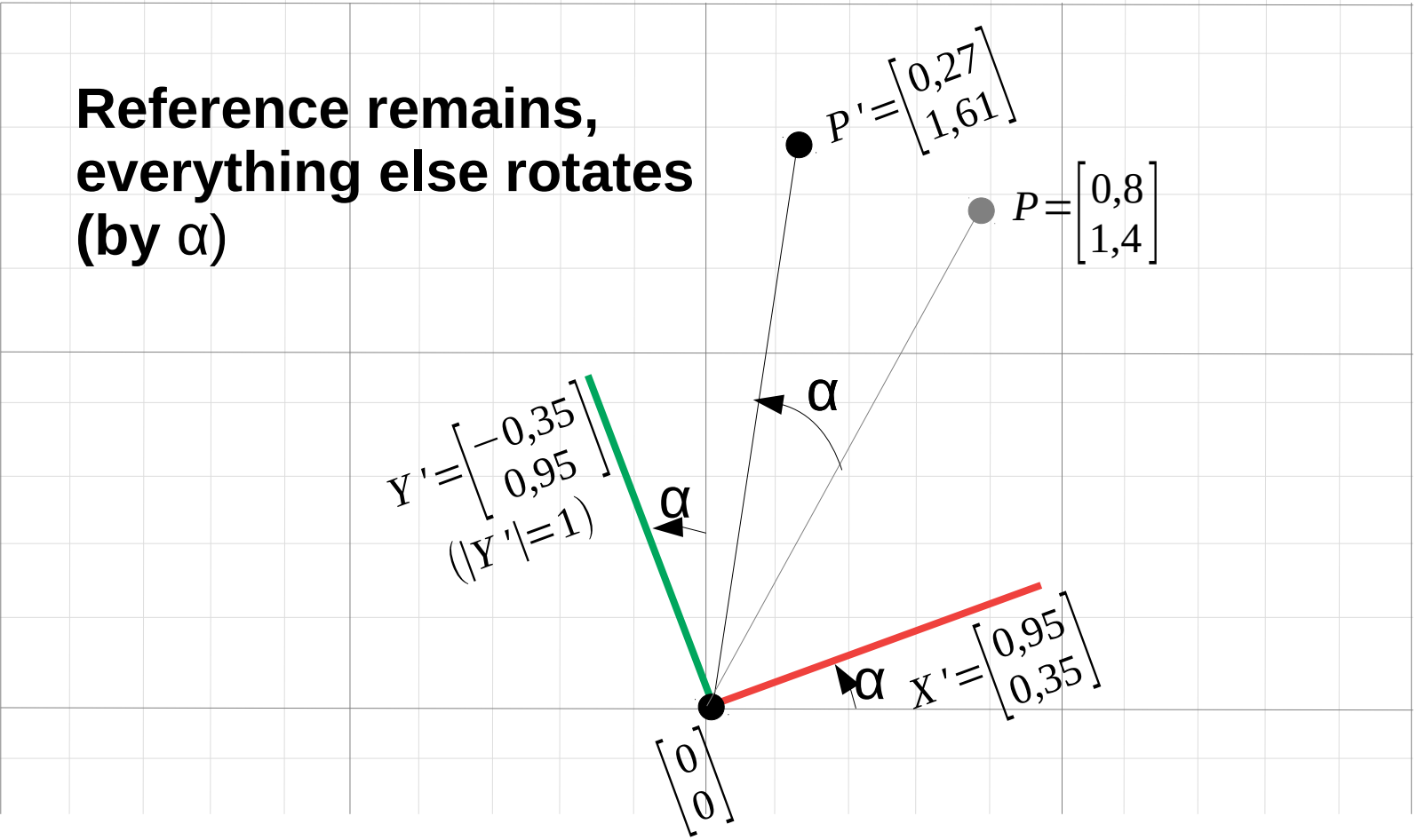
$$[X'Y'] = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

$$P' = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

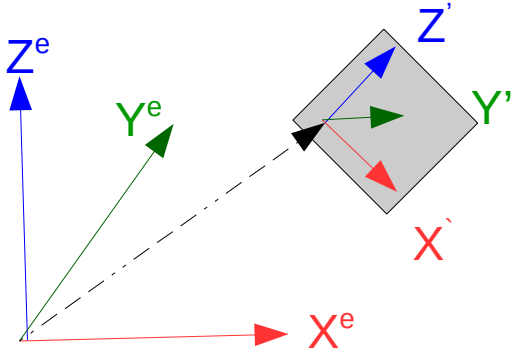
$$Rotor(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

$$P' = Rotor(\alpha) \ * \ P$$

Reference remains, everything else rotates (by α)



For 3D



Reference coordinate system:
- Earth
- ECI
- Inertial
- My initial attitude

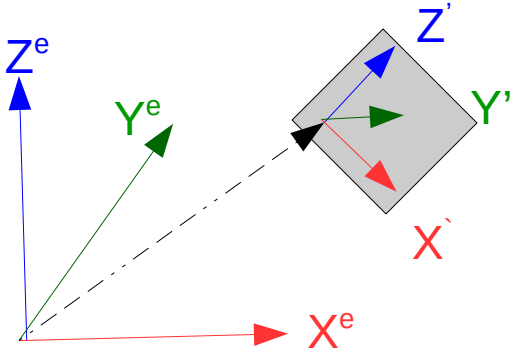
Similarly for 3D:
P = (a, b, c)
P' = a*X' + b*Y' + c*Z'

X' = where will vector (1, 0, 0) strike after rotation
Y' = where will vector (0, 1, 0) strike after rotation
Z' = where will vector (0, 0, 1) strike after rotation

For 3D

Similarly for 3D:
P = (a, b, c)
P' = a*X' + b*Y' + c*Z'

X' = where will vector (1, 0, 0) strike after rotation
Y' = where will vector (0, 1, 0) strike after rotation
Z' = where will vector (0, 0, 1) strike after rotation



Reference coordinate system:
Earth
ECI
Inertial
My initial attitude
origin remains at (0 0 0)

$$P=\begin{bmatrix} p1 \\ p2 \\ p3 \end{bmatrix} \qquad P'=\begin{bmatrix} p1' \\ p2' \\ p3' \end{bmatrix}$$

$$X'=\begin{bmatrix} x1' \\ x2' \\ x3' \end{bmatrix}$$

where will X (originally vector (1, 0, 0)) strike after rotation

$$Y'=\begin{bmatrix} y1' \\ y2' \\ y3' \end{bmatrix}$$

$$Z'=\begin{bmatrix} z1' \\ z2' \\ z3' \end{bmatrix}$$

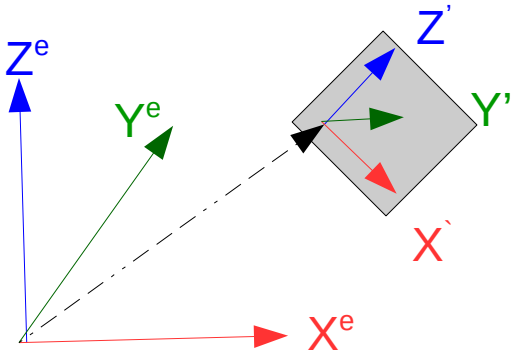
Colours: xzy, on the reference coordinate system

Index 1,2,3 -> xyz on the ref. coord.

For 3D

Similarly for 3D:
P = (a, b, c)
P' = a*X' + b*Y' + c*Z'

X' = where will vector (1, 0, 0) strike after rotation
Y' = where will vector (0, 1, 0) strike after rotation
Z' = where will vector (0, 0, 1) strike after rotation



Reference coordinate system:
Earth
ECI
Inertial
My initial attitude
origin remains at (0 0 0)

$$P=\begin{bmatrix} p1 \\ p2 \\ p3 \end{bmatrix} \qquad P'=\begin{bmatrix} p1' \\ p2' \\ p3' \end{bmatrix}$$

$$X'=\begin{bmatrix} x1' \\ x2' \\ x3' \end{bmatrix}$$

where will X (vector (1, 0, 0)) strike after rotation

$$Y'=\begin{bmatrix} y1' \\ y2' \\ y3' \end{bmatrix}$$

$$Z'=\begin{bmatrix} z1' \\ z2' \\ z3' \end{bmatrix}$$

$$P'=\begin{bmatrix} p1' \\ p2' \\ p3' \end{bmatrix}=[p1*X'+p2*Y'+p3*Z']$$

$$P'=\left[p1*\begin{bmatrix} x1' \\ x2' \\ x3' \end{bmatrix}+p2*\begin{bmatrix} y1' \\ y2' \\ y3' \end{bmatrix}+p3*\begin{bmatrix} z1' \\ z2' \\ z3' \end{bmatrix}\right]$$

$$P' = \begin{bmatrix} p1' \\ p2' \\ p3' \end{bmatrix} = \begin{bmatrix} p1 * X' + p2 * Y' + p3 * Z' \end{bmatrix}$$

$$P' = \begin{bmatrix} p1 * \begin{bmatrix} x1' \\ x2' \\ x3' \end{bmatrix} + p2 * \begin{bmatrix} y1' \\ y2' \\ y3' \end{bmatrix} + p3 * \begin{bmatrix} z1' \\ z2' \\ z3' \end{bmatrix} \end{bmatrix}$$

$$P' = \begin{bmatrix} p1 * x1' + p2 * y1' + p3 * z1' \\ p1 * x2' + p2 * y2' + p3 * z2' \\ p1 * x3' + p2 * y3' + p3 * z3' \end{bmatrix}$$

$$Rotationsmatrix = \begin{bmatrix} X' & Y' & Z' \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} x1' \\ x2' \\ x3' \end{bmatrix} \begin{bmatrix} y1' \\ y2' \\ y3' \end{bmatrix} \begin{bmatrix} z1' \\ z2' \\ z3' \end{bmatrix} \end{bmatrix} = \begin{bmatrix} x1' & y1' & z1' \\ x2' & y2' & z2' \\ x3' & y3' & z3' \end{bmatrix}$$

$$P' = rotation\,\text{matrix} \ * \ P$$

$$P_{refcor.} = Attitude * P_{bodyframe}$$

I define now!

$$P' = \begin{bmatrix} x1' & y1' & z1' \\ x2' & y2' & z2' \\ x3' & y3' & z3' \end{bmatrix} * \begin{bmatrix} p1 \\ p2 \\ p3 \end{bmatrix}$$

$$P' = \begin{bmatrix} p1 * x1' + p2 * y1' + p3 * z1' \\ p1 * x2' + p2 * y2' + p3 * z2' \\ p1 * x3' + p2 * y3' + p3 * z3' \end{bmatrix}$$

$$\boxed{P' = \begin{bmatrix} x1' & y1' & z1' \\ x2' & y2' & z2' \\ x3' & y3' & z3' \end{bmatrix} * \begin{bmatrix} p1 \\ p2 \\ p3 \end{bmatrix}}$$

Is it really the same?

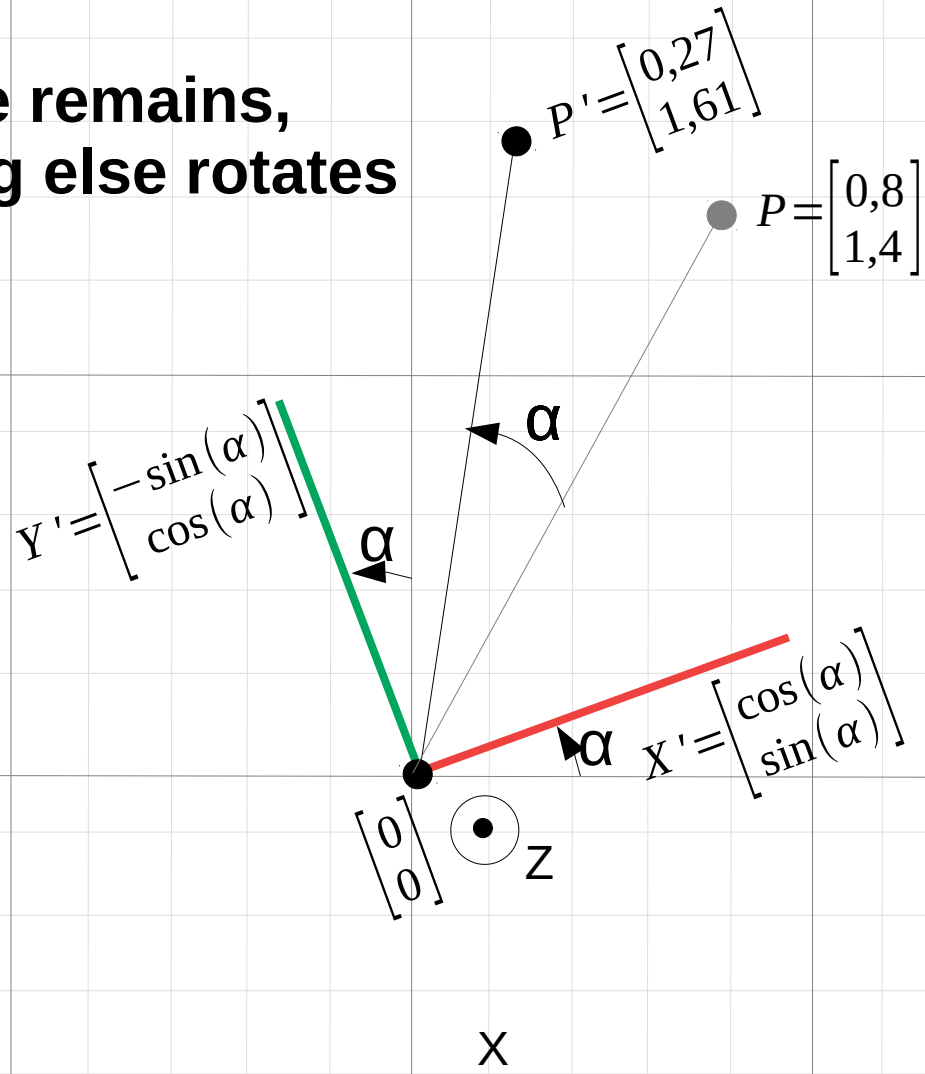
$$P' = \begin{bmatrix} x1' & y1' & z1' \\ x2' & y2' & z2' \\ x3' & y3' & z3' \end{bmatrix} * \begin{bmatrix} p1 \\ p2 \\ p3 \end{bmatrix} \qquad P' = \begin{bmatrix} x1' * p1 + y1' * p2 + z1' * p3 \\ x2' * p1 + y2' * p2 + z2' * p3 \\ x3' * p1 + y3' * p2 + z3' * p3 \end{bmatrix}$$

Remarks

X’, Y’ and Z’ must have length 1,
otherwise multiplied vectors, will be elongated

X’ Y’ and Z’
shall be orthogonal,
otherwise ist will not be equally rotated in all directions

Reference remains, everything else rotates (by α)



For 3D

By the way

Z coming towards us from paper

Z going from us towards paper

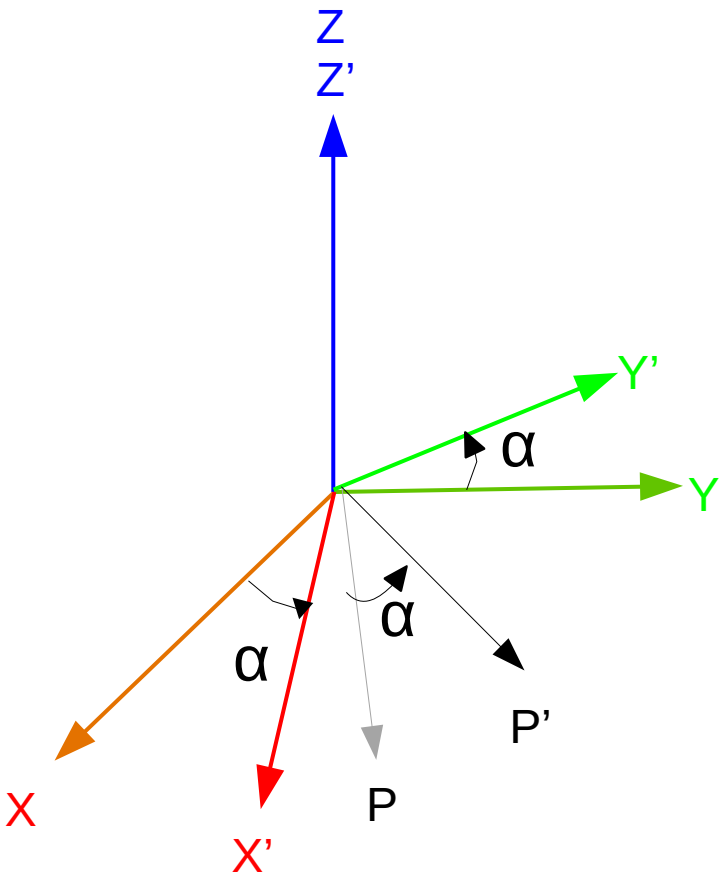
But only in 2D!
Which axis lacks?
It does not lack! Everything was turned around this axis (Z).

$$P' = \begin{bmatrix} x1' & y1' & z1' \\ x2' & y2' & z2' \\ x3' & y3' & z3' \end{bmatrix} * \begin{bmatrix} p1 \\ p2 \\ p3 \end{bmatrix}$$

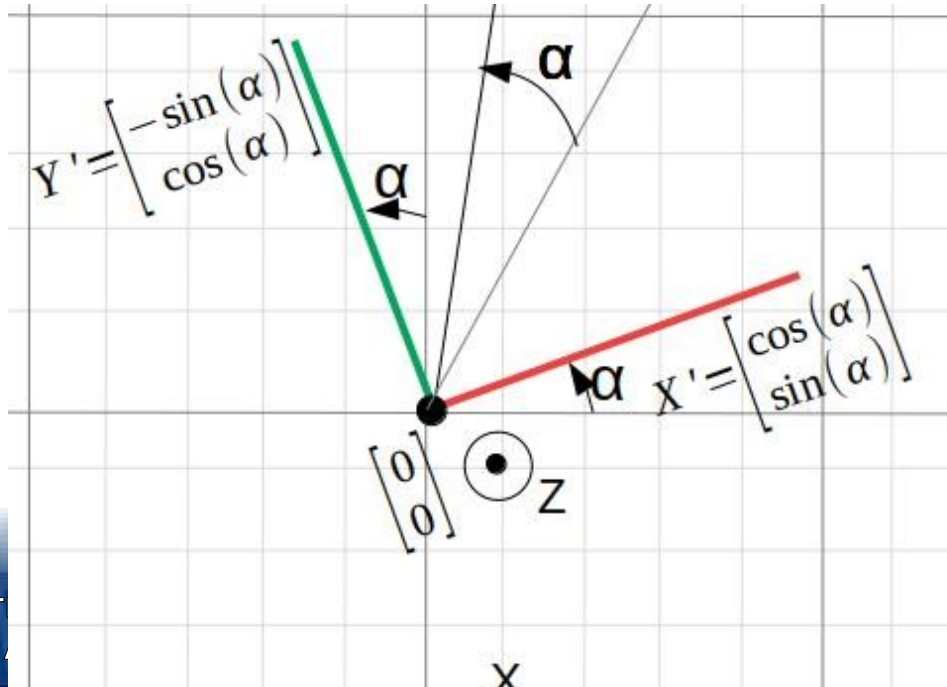
$$X' = \begin{bmatrix} \cos \\ \sin \\ ?? \end{bmatrix} \longrightarrow X' = \begin{bmatrix} \cos \\ \sin \\ 0 \end{bmatrix} \qquad Y' = \begin{bmatrix} -\sin \\ \cos \\ 0 \end{bmatrix} \qquad Z' = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

X' remains on the X-Y plane. It does not rise

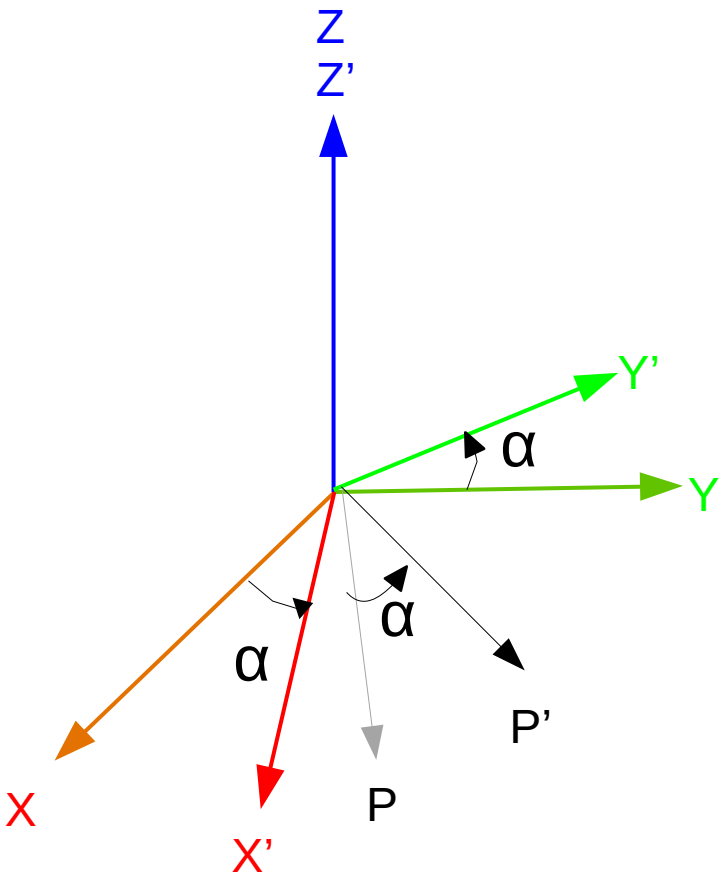
Z' = Z
Z did not change original e_z



$$[X'Y'Z'] = \begin{bmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Arround Z : Rotor(Z, α)



$$Rotor(Z,\alpha)=[X'Y'Z']=\begin{bmatrix}\cos(\alpha) & -\sin(\alpha) & 0\\ \sin(\alpha) & \cos(\alpha) & 0\\ 0 & 0 & 1\end{bmatrix}$$

Here we rotate around Z,
→ Z-coordinates remain the same
This matrix is
Rot(Z, α) * P
If we rotate around Z-axis by α degrees, then

P' = Rot(Z, α) * P

$$Rot(Z,\alpha)=\begin{bmatrix}\cos(\alpha) & -\sin(\alpha) & 0\\ \sin(\alpha) & \cos(\alpha) & 0\\ 0 & 0 & 1\end{bmatrix}$$

P' = Rot(Z , α) * P

$$P'=\begin{bmatrix}p1'\\ p2'\\ p3'\end{bmatrix}=\begin{bmatrix}\cos(\alpha) & -\sin(\alpha) & 0\\ \sin(\alpha) & \cos(\alpha) & 0\\ 0 & 0 & 1\end{bmatrix}*\begin{bmatrix}p1\\ p2\\ p3\end{bmatrix}$$

Rotation 2D → 3D

```
Rotation around Z:   P'  =   Rot(Z,α) *P
Rotation around Y:   P'  =   Rot(Y,ß) *P
Rotation around X:   P'  =   Rot(X,Ψ) *P
```

$$Rot(Z,\alpha)=\begin{bmatrix}\cos(\alpha) & -\sin(\alpha) & 0\\ \sin(\alpha) & \cos(\alpha) & 0\\ 0 & 0 & 1\end{bmatrix}$$

Rotation 2D → 3D

```
Rotation around Z:   P'  =   Rot(Z,α) *P
Rotation around Y:   P'  =   Rot(Y,ß) *P
Rotation around X:   P'  =   Rot(X,Ψ) *P
```

$$Rot(X,\Psi)=\begin{bmatrix}1 & 0 & 1\\ 0 & \cos(\Psi) & -\sin(\Psi)\\ 0 & \sin(\Psi) & \cos(\Psi)\end{bmatrix}$$

$$Rot(Z,\alpha)=\begin{bmatrix}\cos(\alpha) & -\sin(\alpha) & 0\\ \sin(\alpha) & \cos(\alpha) & 0\\ 0 & 0 & 1\end{bmatrix}$$

Rotation 2D → 3D

```
Rotation around Z:   P'  =   Rot(Z,α) *P
Rotation around Y:   P'  =   Rot(Y,ß) *P
Rotation around X:   P'  =   Rot(X,Ψ) *P
```

$$Rot(X,\Psi)=\begin{bmatrix}1 & 0 & 0\\ 0 & \cos(\Psi) & -\sin(\Psi)\\ 0 & \sin(\Psi) & \cos(\Psi)\end{bmatrix}$$

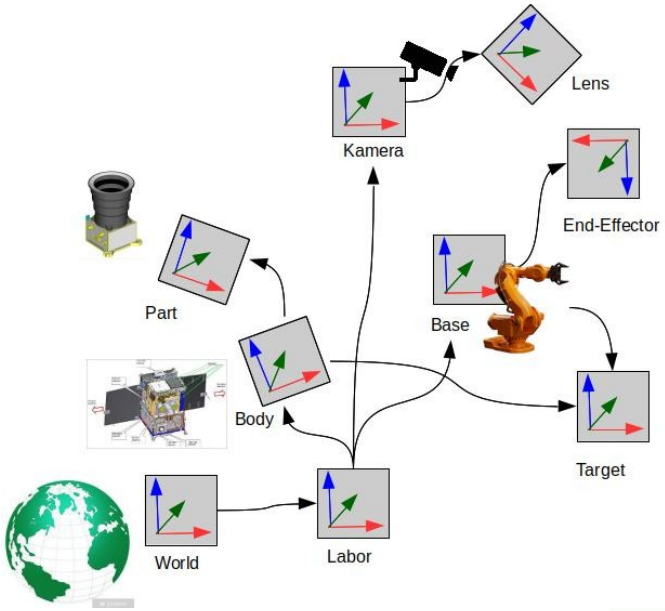
$$Rot(Y,ß)=\begin{bmatrix}\cos(ß) & 0 & \sin(ß)\\ 0 & 1 & 0\\ -\sin(ß) & 0 & \cos(ß)\end{bmatrix}$$

$$Rot(Z,\alpha)=\begin{bmatrix}\cos(\alpha) & -\sin(\alpha) & 0\\ \sin(\alpha) & \cos(\alpha) & 0\\ 0 & 0 & 1\end{bmatrix}$$

The Y anomaly

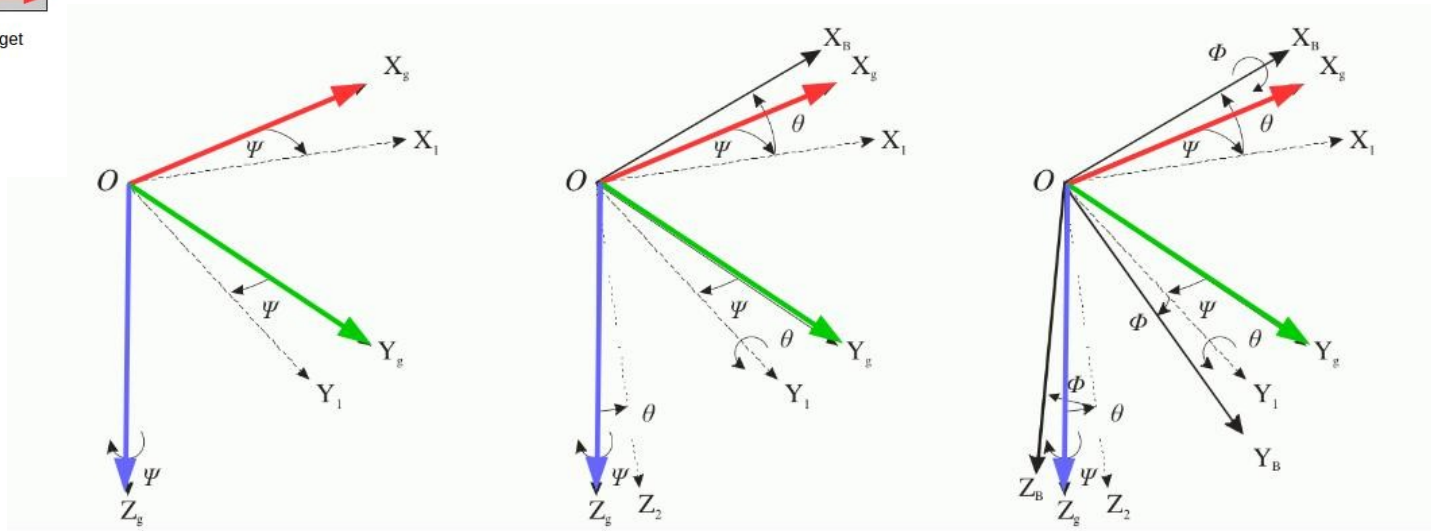
If Y were -Y
then everything would be okay.
(Left handed)
or... rotate the matrix over its boundary

Chains of Rotationen & Accumulation of Rotations



chain

Accumulate



Zuerst Yaw (Z) dann Pitch (Y) Dann Roll (x)

Rotation Matrix

which combines all part-rotations

$$\underline{R} = \begin{pmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{pmatrix}$$

Several rotations by **matrix multiplication**
Remember: right first (from right to left)

IMPORTANT:
Rotations are non-commutative
neither are matrix multiplications
R1 · R2 ≠ R2 · R1

$$Rot\,(x,\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi & \cos\varphi \end{bmatrix}$$

e.g. I rotate around Z Ψ, then around X α and then around Z β:

$$Rot\,(y,\varphi) = \begin{bmatrix} \cos\varphi & 0 & \sin\varphi \\ 0 & 1 & 0 \\ -\sin\varphi & 0 & \cos\varphi \end{bmatrix}$$

corresponding Rotor = Rot(Z, β) * Rot(X, α) * Rot(Z, Ψ)

$$Rot\,(z,\varphi) = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Could I not simply add Ψ and β?
Both are on Z!
See physical example with 3 axes

Remember: Sequence of Rotations

(Same is true for matrices, quaternions, RPY)

1. A
2. B
3. C

LuRis

Transformation of a point: ←

The matrix of the first rotation is most right: CBA

we rotate, the world (reference coordinate system) is fixed

This is the case in

navigation

control (of rotations)

→

Games:

Transformation of coordinate system: (inverse) : ABC

The matrix of the first rotation is most left:

We are fixed, the rest of the universe rotates around us

This is the case in

games, computer graphics: the world rotates around the camera

However ...

to combine

Rot(X, β) then Rot(Y, α) then Rot(Z, Ψ)

intensely looks like RPY (Euler angles)!

... and we said, do not calculate like this for long time

(only small angles)

Yes... Therefore only small steps and than accumulate in an overall rotation matrix

Difference: The complete attitude (big deviations from 0-attitude) is kept in one matrix, not in 3 angles (RPY).
In both cases, small deviations are encoded in 3 angles.

Back and Forth

$$Rot(Z, \alpha)^{-1} = Rot(Z,-\alpha) = Rot(Z,\alpha)^{T}$$

... If all vectors were orthogonal
... They are. We will see

Why?

Rot(Z,α)

$$\begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Rot(Z,-α)

$$\begin{pmatrix} \cos(-\alpha), & -\sin(-\alpha), & 0 \\ \sin(-\alpha), & \cos(-\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix} = \begin{pmatrix} \cos(\alpha), & \sin(\alpha), & 0 \\ -\sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Rot(Z,-α)

invert sines
or
reflect across diagonal
At each: X, Y, Z

Back and Forth

$$\mathrm{Rot}(Z, \alpha)^{-1} = \mathrm{Rot}(Z,-\alpha) = \mathrm{Rot}(Z,\alpha)^{T}$$

... If all vectors were orthogonal
... They are. We will see

$\mathrm{Rot}(Z,\alpha)$

$$\begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

$\mathrm{Rot}(Z,-\alpha)$

$$\begin{pmatrix} \cos(-\alpha), & -\sin(-\alpha), & 0 \\ \sin(-\alpha), & \cos(-\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix} = \begin{pmatrix} \cos(\alpha), & \sin(\alpha), & 0 \\ -\sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

$\mathrm{Rot}(Z,-\alpha)$

For each X, Y, Z : here called A

$$\mathrm{Rot}(A, \beta)^{-1} = \mathrm{Rot}(A, -\beta) = \mathrm{Rot}(A, \beta)^{T}$$

$$\sin(\beta) \rightarrow \sin(-\beta) = -\sin(\beta)$$

$$\cos(\beta) \rightarrow \cos(-\beta) = \cos(\beta)$$

Interesting Informations

transposed or negative angles
(reflected across the diagonale)
^-1 = T, only because vectors are orthogonal

$$ROT_{b\text{->}k} = (\vec{U}, \vec{V}, \vec{W}) = \begin{pmatrix} U_x & V_x & W_x \\ U_y & V_y & W_y \\ U_z & V_z & W_z \end{pmatrix}$$

$$Rot_{b\text{->}k} = (Rot_{k\text{->}b})^{-1} = (Rot_{k\text{->}b})^{t}$$

U, V, W are orthogonal

$U \bullet V = 0$

$V \bullet W = 0$

$U \bullet W = 0$

$U \times V = W$

$V \times W = U$

$U \times W = V$

U,V,W are normalised

$|U| = |V| = |W| = 1$

Really?

Example: Rot(Z,α)

$$\begin{array}{ccc} U & V & W \\ \end{array}$$
$$\begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Are U, V and W orthogonal?
check....

U, V, W are orthogonal

$U \bullet V = 0$
$V \bullet W = 0$
$U \bullet W = 0$

$U \times V = W$
$V \times W = U$
$W \times U = V$

Are U, V and W normalised?
check....

U,V,W are normalised

$|U| = |V| = |W| = 1$

Really?

Example: Rot(Z,α)

$$\begin{array}{ccc} U & V & W \\ \end{array}$$
$$\begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Are U, V and W orthogonal?
check....

U, V, W are orthogonal

U ● V = 0	cos*(-sin) + sin*cos + 0*0 = 0
V ● W = 0	cos*0 + sin*0 + 0*1 = 0
U ● W = 0	-sin*0 + cos*0 + 0*1 = 0

U x V = W
V x W = U
W x U = V

Are U, V and W normalised?
check....

U,V,W are normalised

$|U| = |V| = |W| = 1$
$\cos^2 + \sin^2 + 0 = 1$
$-\sin^2 + \cos^2 + 0 = 1$
$0 + 0 + 1 = 1$

Really?

Example: Rot(Z,α)

$$\begin{array}{ccc} U & V & W \\ \end{array}$$
$$\begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

$$\begin{array}{ccc} \mathbf{U} & \mathbf{X} & \mathbf{V} \\ c & & -s \\ s & & c \\ 0 & & 0 \\ c & & -s \\ s & & c \\ 0 & & 0 \end{array} = \begin{pmatrix} s0-c0 \\ 0(-s)-0c \\ cc-(-s)s \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Are U, V and W orthogonal?
check....

U, V, W are orthogonal
$U \bullet V = 0$
$V \bullet W = 0$
$U \bullet W = 0$

$U \times V = W$
$V \times W = U$
$U \times W = V$

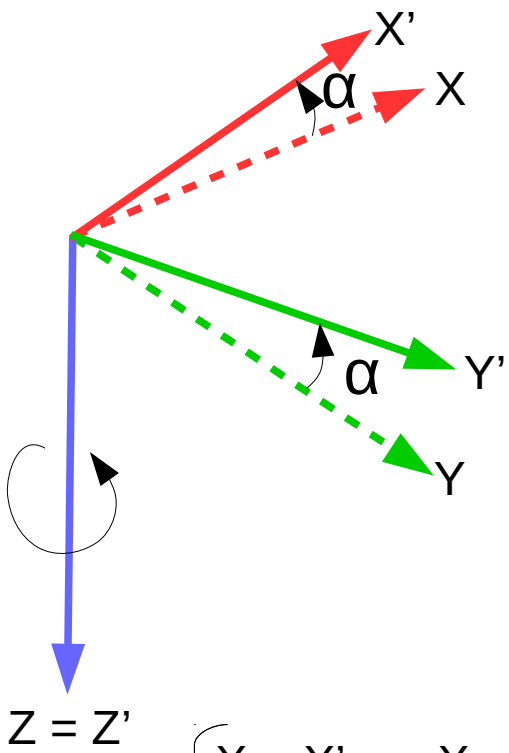
Are U, V and W normalised?
check....

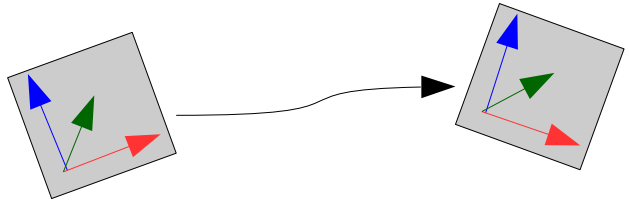
U,V,W are normalised
$|U| = |V| = |W| = 1$
$\cos^2 + \sin^2 + 0 = 1$
$-\sin^2 + \cos^2 + 0 = 1$
$0 + 0 + 1 = 1$

$$\begin{array}{ccc} \mathbf{W} & \mathbf{X} & \mathbf{U} \\ 0 & & c \\ 0 & & s \\ 1 & & 0 \\ 0 & & c \\ 0 & & s \\ 1 & & 0 \end{array} = \begin{pmatrix} 00-1s \\ 1c-00 \\ 0s-0c \end{pmatrix} = \begin{pmatrix} -s \\ c \\ 0 \end{pmatrix}$$

Let's Check the Direction Cosinus Matrix

Let's say, we rotate only around Z, by α





(x, y, z) → (x', y', z')

Z=Z'
Z=Z' remains orthogonal to x, x' and y, y'

Scalar product of orthogonal vectors = 0

Scalar product of a ● a = 1

cos(90) = 0
cos(0) = 1
cos(α + 90) = -sin(α)
cos(90 - α) = sin(α)
from X to Y = 90 grad
from X' to Y' still 90 degrees

$$\begin{pmatrix} X \bullet X' & X \bullet Y' & X \bullet Z' \\ Y \bullet X' & Y \bullet Y' & Y \bullet Z' \\ Z \bullet X' & Z \bullet Y' & Z \bullet Z' \end{pmatrix} \rightarrow \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Correct?

Reminder
Matrix Multiplikation

$$\begin{pmatrix} m11 & m12 & m13 \\ m21 & m22 & m23 \\ m31 & m32 & m33 \end{pmatrix} * \begin{pmatrix} p11 & p12 & p13 \\ p21 & p22 & p23 \\ p31 & p32 & p33 \end{pmatrix} =$$

$$\begin{pmatrix} m11p11 + m12p21 + m13p31 & m11p12 + m12p22 + m13p32 & m11p13 + m12p23 + m13p33 \\ m21p11 + m22p21 + m23p31 & m21p12 + m22p22 + m23p32 & m21p13 + m22p23 + m23p33 \\ m31p11 + m32p21 + m23p31 & m31p12 + m32p22 + m33p32 & m31p13 + m32p23 + m33p33 \end{pmatrix}$$

Reminder
Matrix Multiplikation

Scalar product of these two vectors

$$\begin{pmatrix} m11 & m12 & m13 \\ m21 & m22 & m23 \\ m31 & m32 & m33 \end{pmatrix} * \begin{pmatrix} p11 & p12 & p13 \\ p21 & p22 & p23 \\ p31 & p32 & p33 \end{pmatrix} =$$

$$\begin{pmatrix} m11p11 + m12p21 + m13p31 & m11p12 + m12p22 + m13p32 & m11p13 + m12p23 + m13p33 \\ m21p11 + m22p21 + m23p31 & m21p12 + m22p22 + m23p32 & m21p13 + m22p23 + m23p33 \\ m31p11 + m32p21 + m23p31 & m31p12 + m32p22 + m33p32 & m31p13 + m32p23 + m33p33 \end{pmatrix}$$

$$\begin{pmatrix} A \\ B \\ C \end{pmatrix} * \begin{pmatrix} K & L & M \end{pmatrix} = \begin{pmatrix} AK & AL & AM \\ BK & BL & BM \\ CK & CL & CM \end{pmatrix}$$

A B C K L M : 3D Vektoren AK: Skalarprodukt

Concatenated Rotations

Rot(α, β, Ψ) = Rot(x, α) * Rot(y, β) * Rot(z, Ψ) (RPY)

Non permutative! Pay attention what will be left/right
Assuming very small steps and aviation norm
RPY = X*Y*Z (first Z, then Y, then X)

$$R_x R_y R_z$$

$$= \begin{pmatrix} 1 & 0 & 0 \\ 0 & cos\theta_x & -sin\theta_x \\ 0 & sin\theta_x & cos\theta_x \end{pmatrix} \begin{pmatrix} cos\theta_y & 0 & sin\theta_y \\ 0 & 1 & 0 \\ -sin\theta_y & 0 & cos\theta_y \end{pmatrix} \begin{pmatrix} cos\theta_z & -sin\theta_z & 0 \\ sin\theta_z & cos\theta_z & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} cos\theta_y cos\theta_z & -cos\theta_y sin\theta_z & sin\theta_y \\ cos\theta_x sin\theta_z + sin\theta_x sin\theta_y cos\theta_z & cos\theta_x cos\theta_z - sin\theta_x sin\theta_y sin\theta_z & -sin\theta_x cos\theta_y \\ sin\theta_x sin\theta_z - cos\theta_x sin\theta_y cos\theta_z & sin\theta_x cos\theta_z + cos\theta_x sin\theta_y sin\theta_z & cos\theta_x cos\theta_y \end{pmatrix}$$

where $\theta_x = tilt\ about\ x, \theta_y = tilt\ about\ y, \theta_z = tilt\ about\ z$

Concatenated Rotations

$$Rot(\alpha, \beta, \Psi) = Rot(x, \alpha) * Rot(y, \beta) * Rot(z, \Psi) \qquad (RPY)$$

Concatenation:

A then B then C, total D (sequence of rotations)

And back: D^{-1} :

Concatenated Rotations

$$Rot(\alpha, \beta, \Psi) = Rot(x, \alpha) * Rot(y, \beta) * Rot(z, \Psi) \qquad (RPY)$$

Concatenation:

A then B then C, total D (sequence of rotations)

$D = C*B*A$

And back: D^{-1} : $A^{-1} * B^{-1} * C^{-1}$

first reverse C, then reverse B, and then reverse A

Concatenated Rotations

Concatenation and sequences of transformations (6 DoF)

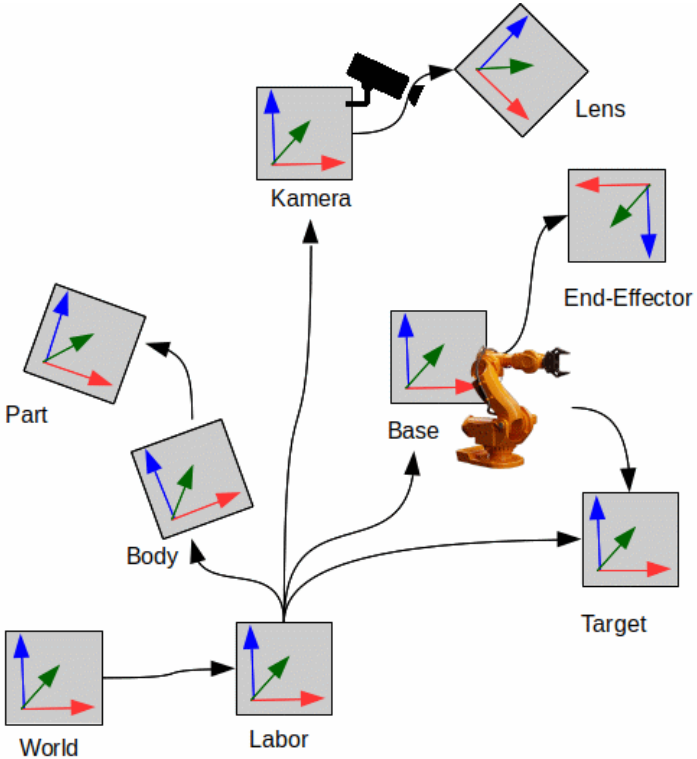
$B \rightarrow K \quad = \quad B \rightarrow L$ and then $L \rightarrow K$

e.g. attitude of camera relative to attitude of Base
…
Base to camera via Lab

second rotation first rotation

$$\mathrm{Rot}(B \rightarrow K) = \mathrm{Rot}(L \rightarrow K) \quad \mathrm{Rot}(B \rightarrow L)$$

Please from right to left



e.g. two Rotations of the same Axis

Rot(Z,α+ß) ? = Rot(Z,α) * Rot(Z, ß)

$$\begin{pmatrix} \cos(\alpha+ß), & -\sin(\alpha+ß), & 0 \\ \sin(\alpha+ß), & \cos(\alpha+ß), & 0 \\ 0, & 0, & 1 \end{pmatrix} \overset{?}{=} \begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix} * \begin{pmatrix} \cos(ß), & -\sin(ß), & 0 \\ \sin(ß), & \cos(ß), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Rot(Z, α + ß)

$$\begin{pmatrix} \cos(\alpha)*\cos(ß) - \sin(\alpha)*\sin(ß), & -\cos(\alpha)*\sin(ß) - \sin(\alpha)*\cos(ß), & 0 \\ \sin(\alpha)*\cos(ß) + \cos(\alpha)*\sin(ß), & -\sin(\alpha)*\sin(ß) - \cos(\alpha)*\cos(ß), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

e.g. two Rotations of the same Axis

Rot(Z,α+ß) Rot(Z,α) Rot(Z, ß)

$$\begin{pmatrix} \cos(\alpha+ß), & -\sin(\alpha+ß), & 0 \\ \sin(\alpha+ß), & \cos(\alpha+ß), & 0 \\ 0, & 0, & 1 \end{pmatrix} \overset{?}{=} \begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix} * \begin{pmatrix} \cos(ß), & -\sin(ß), & 0 \\ \sin(ß), & \cos(ß), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Rot(Z, α + ß)

cos(α)*cos(ß) - sin(α)*sin(ß),	-cos(α)*sin(ß) - sin(α)*cos(ß),	0
sin(α)*cos(ß) + cos(α)*sin(ß),	-sin(α)*sin(ß) - cos(α)*cos(ß),	0
0,	0,	1

Reminder from textbook

cos(α + β) = - sin (α) * sin(β) + cos (α) * cos(β)
sin (α + β) = sin (α) * cos(β) + cos (α) * sin(β)

e.g. two Rotations of the same Axis

Rot(Z,α+ß) Rot(Z,α) Rot(Z, ß)

$$\begin{pmatrix} \cos(\alpha+ß), & -\sin(\alpha+ß), & 0 \\ \sin(\alpha+ß), & \cos(\alpha+ß), & 0 \\ 0, & 0, & 1 \end{pmatrix} \overset{?}{=} \begin{pmatrix} \cos(\alpha), & -\sin(\alpha), & 0 \\ \sin(\alpha), & \cos(\alpha), & 0 \\ 0, & 0, & 1 \end{pmatrix} * \begin{pmatrix} \cos(ß), & -\sin(ß), & 0 \\ \sin(ß), & \cos(ß), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Rot(Z, α + ß)

$$\begin{pmatrix} \cos(\alpha)*\cos(ß) - \sin(\alpha)*\sin(ß), & -\cos(\alpha)*\sin(ß) - \sin(\alpha)*\cos(ß), & 0 \\ \sin(\alpha)*\cos(ß) + \cos(\alpha)*\sin(ß), & -\sin(\alpha)*\sin(ß) - \cos(\alpha)*\cos(ß), & 0 \\ 0, & 0, & 1 \end{pmatrix}$$

Reminder from textbook

$\cos(\alpha + \beta) = -\sin(\alpha) * \sin(\beta) + \cos(\alpha) * \cos(\beta)$

$\sin(\alpha + \beta) = \sin(\alpha) * \cos(\beta) + \cos(\alpha) * \sin(\beta)$

Siehe
056b-rotationen-mathe.cpp

Concatenation Developing a Program

Several rotations by **matrix multiplication**

Vector V is rotated like in A (matrix), then like in B, result W
A,B,C are 3x3 Matrizen, V and W are 3D vectors

W = A * V first A then B and then C!

subsequently around B and then around C

W = B * W
W = C * W

The entire rotation matrix (e.g. for all 3 axes) is prepared

Or around A and then around B and then C

W = C * (B * (A * V)) = (C*B*A) * V

The corresponding rotation matrix

D = C * (B * A)
W = D * V

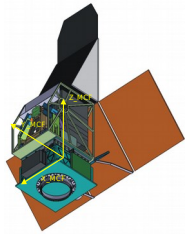
If we do not rotate the coordinate system, then the other way round:

D = ((A*B) * C)

Calculation of Rotations

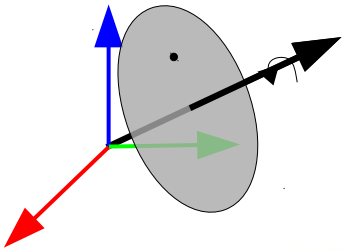
Rotation matrix

Quaternions (rotation vector, rotation angle)



Euler-Angle (incl. RPY)

The direct path from A to B
without considering the axes X, Y, Z



Why complex numbers?

H - Quaternion: Definition

$$q \in R^4$$

$$q_0 \in R$$

$$\mathbf{q} \in R^3 \cdot i$$

$$q = \boxed{q_0 + \vec{\mathbf{q}}} = \boxed{q_0 + \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}}$$

ZB $1 \rightarrow x$
$2 \rightarrow y$
$3 \rightarrow z$

$$|q| = \sqrt[2]{q_0^2 + q_1^2 + q_2^2 + q_3^2} \quad \text{(KEIN } i\text{)}$$

EINHEITS QUATERNION $|q| = 1$

3 orthogonale imaginary axes

Why complex numbers?

Quaternion Definition

Modern, what I use:

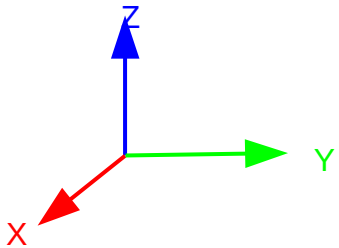
$$Q = \left(q0 + \begin{bmatrix} x \\ y \\ z \end{bmatrix} \right)$$

Original by Hamilton and Rodriguez:

- $i^2 = j^2 = k^2 = -1$
- $ij = +k, \quad jk = +i, \quad ki = +j$
- $ji = -k, \quad kj = -i, \quad ik = -j$

$$Q = x_0 + x_1 i + x_2 j + x_3 k$$

Same as (vector product)
xy = z yx = -z
yz = x zy = -x
zx = y xz = -y
But
xx = yy = zz = (000) !
But then again
-1 does not exist
in (*i j k*) there it is (0i 0j 0k) -1



Attention

Always normalised!

Initialisise: ATTENTION! never with (0, (0,0,0)) !
zero rotation: (1, (0,0,0))
around vector (0,0,0) ?? is ok... we will see

Beware:
Don‘t try to normalise (0,0,0) !
This is a zero-rotation.
Always catch it with if(...) !

Rotations with complex numbers

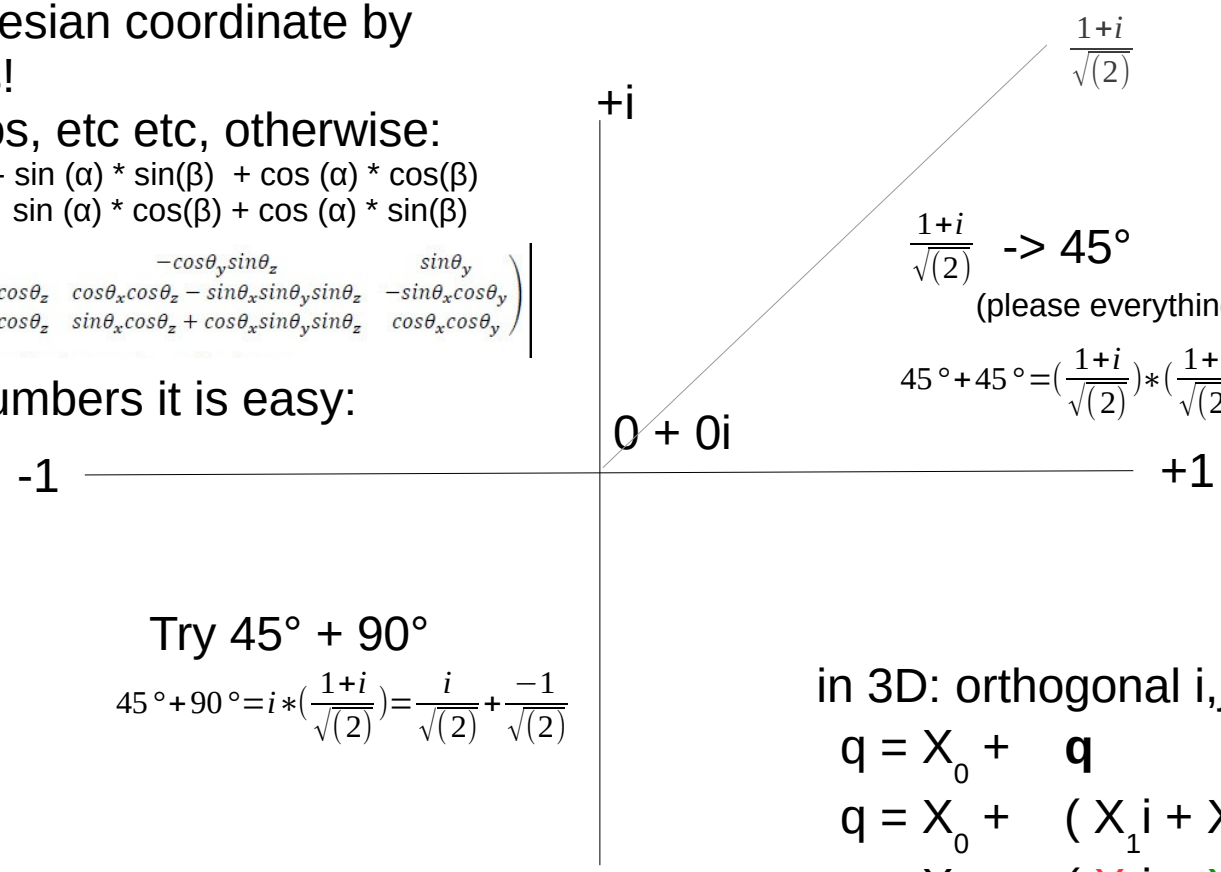
Auch in 056a-rotationen.odp

Rotate in Cartesian coordinate by multiplications!
without sin, cos, etc etc, otherwise:

cos(α + β) = - sin (α) * sin(β) + cos (α) * cos(β)
sin (α + β) = sin (α) * cos(β) + cos (α) * sin(β)

$$\begin{pmatrix} cos\theta_y cos\theta_z & -cos\theta_y sin\theta_z & sin\theta_y \\ cos\theta_x sin\theta_z + sin\theta_x sin\theta_y cos\theta_z & cos\theta_x cos\theta_z - sin\theta_x sin\theta_y sin\theta_z & -sin\theta_x cos\theta_y \\ sin\theta_x sin\theta_z - cos\theta_x sin\theta_y cos\theta_z & sin\theta_x cos\theta_z + cos\theta_x sin\theta_y sin\theta_z & cos\theta_x cos\theta_y \end{pmatrix}$$

by complex numbers it is easy:
v1 * v2



$\frac{1+i}{\sqrt{(2)}}$ -> 45°

(please everything normalised)

$$45° + 45° = \left(\frac{1+i}{\sqrt{(2)}}\right) * \left(\frac{1+i}{\sqrt{(2)}}\right) = i = 90°$$

1 + i *
1 + i
_____
-1 + 2i + 1
= 2i

Try 45° + 90°

$$45° + 90° = i * \left(\frac{1+i}{\sqrt{(2)}}\right) = \frac{i}{\sqrt{(2)}} + \frac{-1}{\sqrt{(2)}}$$

in 3D: orthogonal i,j,k

$q = X_0 + \mathbf{q}$

$q = X_0 + (X_1 i + X_2 j + X_3 k)$

$q = X_0 + (X_1 i + X_2 j + X_3 k)$

$i^2 = j^2 = k^2 = ijk = -1$

!!!

-1 * 1 = 180°
i * 1 = 90°
-> Rotate by multiplication

Rotations with i, but without i :)

Angle encoded as complex number: a + b*i, or let‘s say (a,b)
I write a +b*i, but I cannot add them! They stay apart

e.g.

$a_0 + a_1$ a_0 is real part, a_1 is imaginary part

complex multiplication:

$$(a_0 + a_1)*(b_0 + b_1) = \boxed{a_0 * b_0} + \boxed{a_0 * b_1} + \boxed{a_1 * b_0} - \boxed{a_1 b_1}$$

$c = a*b$

$c_0 = a_0 * b_0 - a_1 * b_1$ Real part

$c_1 = a_1 * b_0 + a_0 * b_1$ Imaginary part

Real

Imaginary

Now forget those i-s ....
we needeed them only in order
to define this multiplication

and now with 3D i

(Now forget those i-s)

Quaternion: Definition

angle here as cos: (α)
(not the angle... but... 1/2)

rotation vector there (a)

$$q \in R^4$$

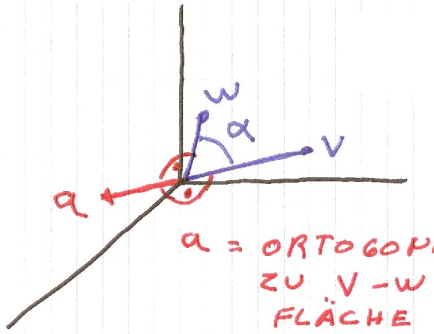
$$q_0 \in R$$

$$\mathbf{q} \in R^3 \cdot i$$

$$q = \boxed{q_0 + \mathbf{q}} = \boxed{q_0 + \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}}$$

zB 1 → x
2 → y
3 → z

q = w X v



$q_0 = \cos(\alpha/2) = f(w \cdot v)$; f = ??

Why?

(Now forget those i-s)

Quaternion: Definition

angle here as cos: (α)
(not the angle... but... 1/2))

rotation vector there (a)

$$q \in R^4$$

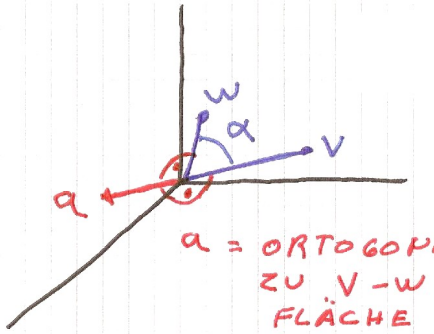
$$q_0 \in R$$

$$\mathbf{q} \in R^3 \cdot i$$

$$q = \boxed{q_0 + \mathbf{q}} = \boxed{q_0 + \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}}$$

zB 1 → x
2 → y
3 → z

q = w X v



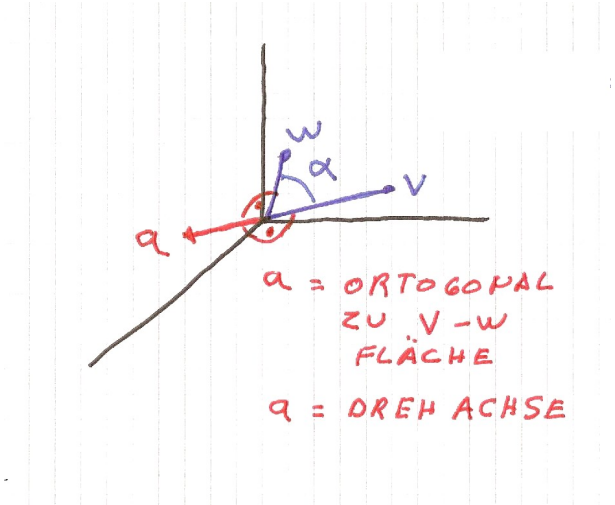
$q_0 = \cos(\alpha/2) = f(w \cdot v)$; f = ??

Why?

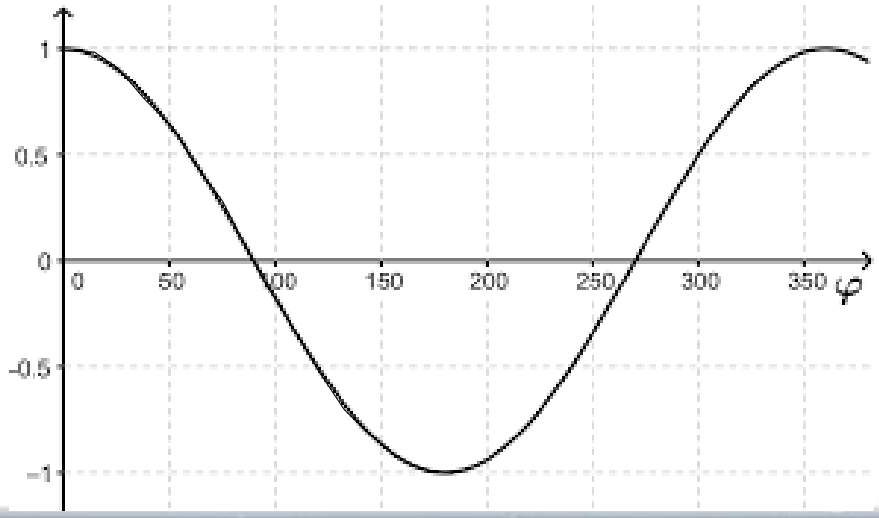
f = cos(acos(w.v)/2)

Quaternion from v to w

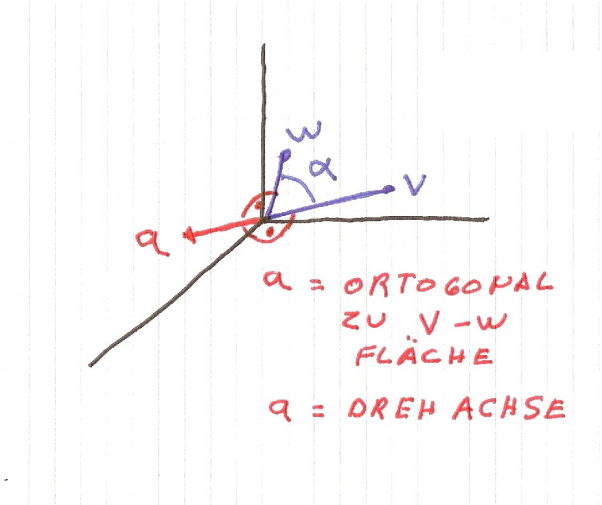
a = w X v
cos(α) = w ● v



But! cos(α) von 0° .. 360°
is ambiguous



Quaternion from v to w



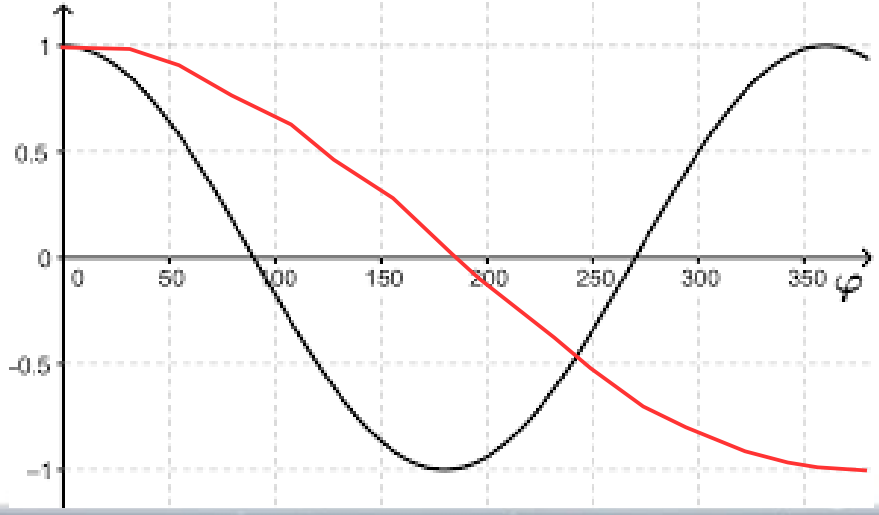
a = w X v
cos(α) = w ● v

Quaternion:

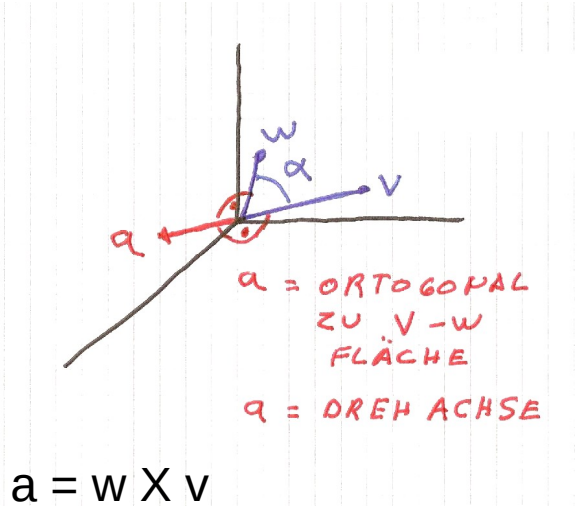
$q_0 = \cos(\alpha/2)$

$\mathbf{q} = a/|a| * ???$

in order to normalise it



Quaternion Def.



a = w X v
cos(α) = w ● v

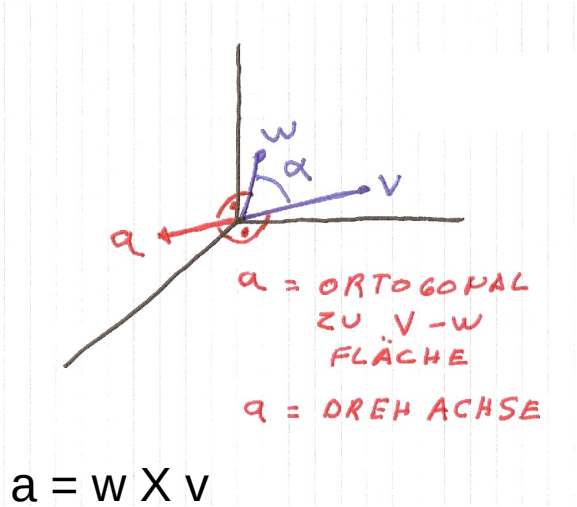
$$q = q_0 + \mathbf{q} = q_0 + \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}$$

$$q = \underbrace{\cos\frac{\alpha}{2}}_{q_0} + \underbrace{\frac{\vec{a}}{|\vec{a}|}}_{\mathbf{q}}$$

* ???

Normalised please! (q, not only **q**)

Quaternion Def.



a = w X v
cos(α) = w ● v

$$q = q_0 + \mathbf{q} = q_0 + \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}$$

$$q = \underbrace{\cos\frac{\alpha}{2}}_{q_0} + \underbrace{\frac{\vec{a}}{|\vec{a}|} \cdot \sin\frac{\alpha}{2}}_{\mathbf{q}}$$

q * sin : in order to normalise entire Q
(otherwise we modify the length of a)

Rotation around 0

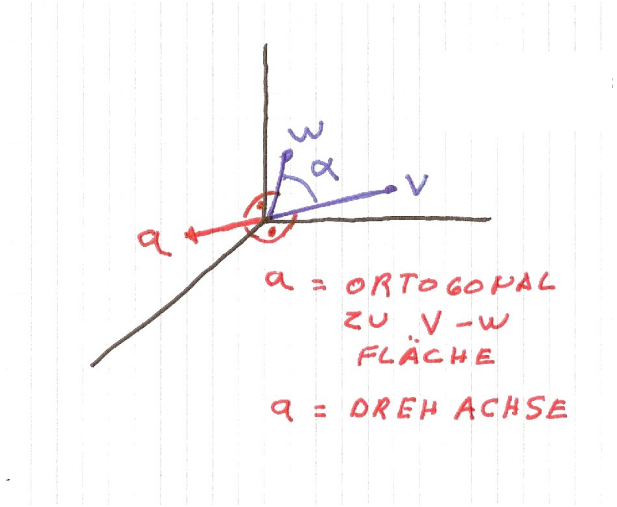
$Q = (\cos(\alpha/2), (x,y,z)*\sin(\alpha/2))$

$Q = (1, (0,0,0))$

Rotation around 0 around any axis!
actually self-evident! or not?

Quaternion from v to w

a = w X v
cos(α) = w ● v



Quaternion from v to w:

Rotation axis : a = w X v (v is right, we work from v to w)
Rotation angle: α = acos(w●v)

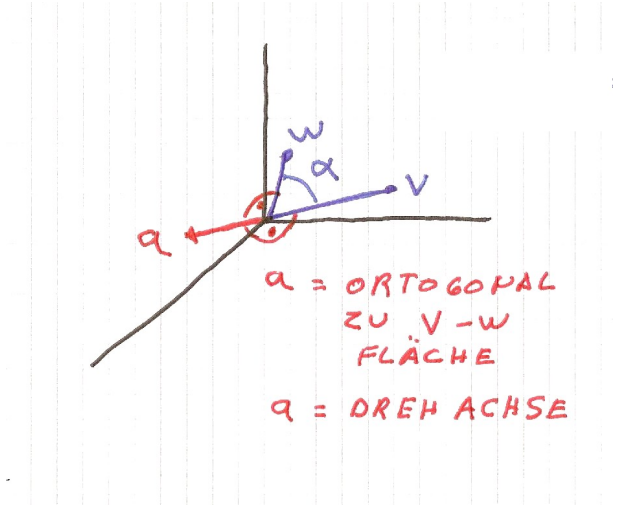
Quaternion:

q_0 = cos(α/2) =
q = a/|a| * sin(α/2) =

as function of v and w

Quaternion from v to w

a = w X v
cos(α) = w ● v



Quaternion from v to w:

Rotation axis : a = w X v (v is right, we work from v to w)
Rotation angle: α = acos(w●v)

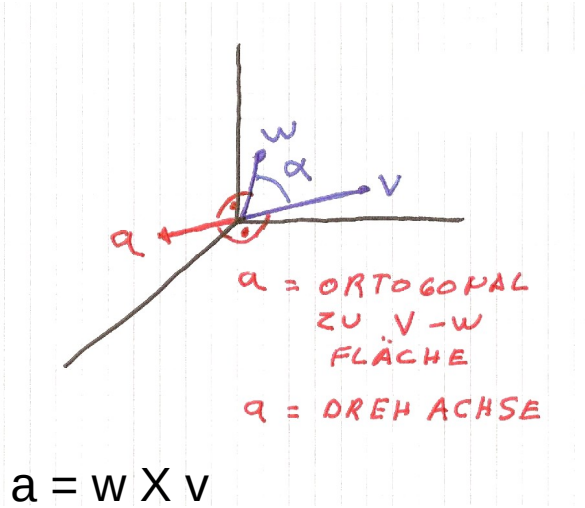
Quaternion (Q):

q_0 = cos(α/2) = cos(acos(w●v)/2)

q = a/|a| * sin(α/2) = wXv / |wXv| * sin(acos(w●v)/2)

Did I say, without sin/cos?
... not so many :)

Rotation with Quaternions



a = w X v
cos(α) = w ● v

What kind of multiplication it that?
4D with complex numbers

$$\vec{w} = q \odot \vec{v} \odot q^*$$

$\mathbb{R}^3$ $\mathbb{R}^4$ $\mathbb{R}^3$ $\mathbb{R}^4$

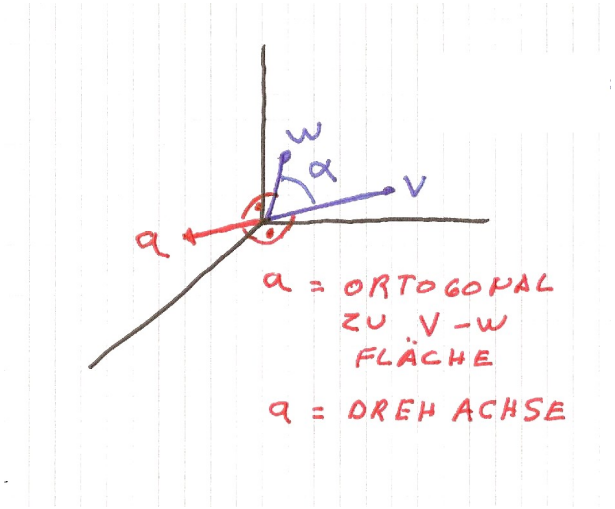
COMPLEX-CONJUGIERT

ROTATIONS-QUATERNION

$$q = q_0 + \mathbf{q} = q_0 + \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}$$

$$q = \underbrace{\cos\frac{\alpha}{2}}_{q_0} + \underbrace{\frac{\vec{a}}{|\vec{a}|} \cdot \sin\frac{\alpha}{2}}_{\mathbf{q}}$$

Rotation with Quaternions



$$\overset{\mathbb{R}^3}{\vec{w}} = \overset{\mathbb{R}^4}{q} \odot \overset{\mathbb{R}^3}{\vec{v}} \odot \overset{\mathbb{R}^4}{q^*}$$

COMPLEX-CONJUGIERT

ROTATIONS-QUATERNION

$\odot$ Later...

= Clifford product of two normalized (**4D**) vectors

$$\overset{\mathbb{R}^3}{\vec{v}} \rightarrow \overset{\mathbb{R}^4}{(0 + \vec{v})}$$

Q and Q^* (Q^* = Q congruent)

Q^* = Mutual rotation of Q, or: rotate back

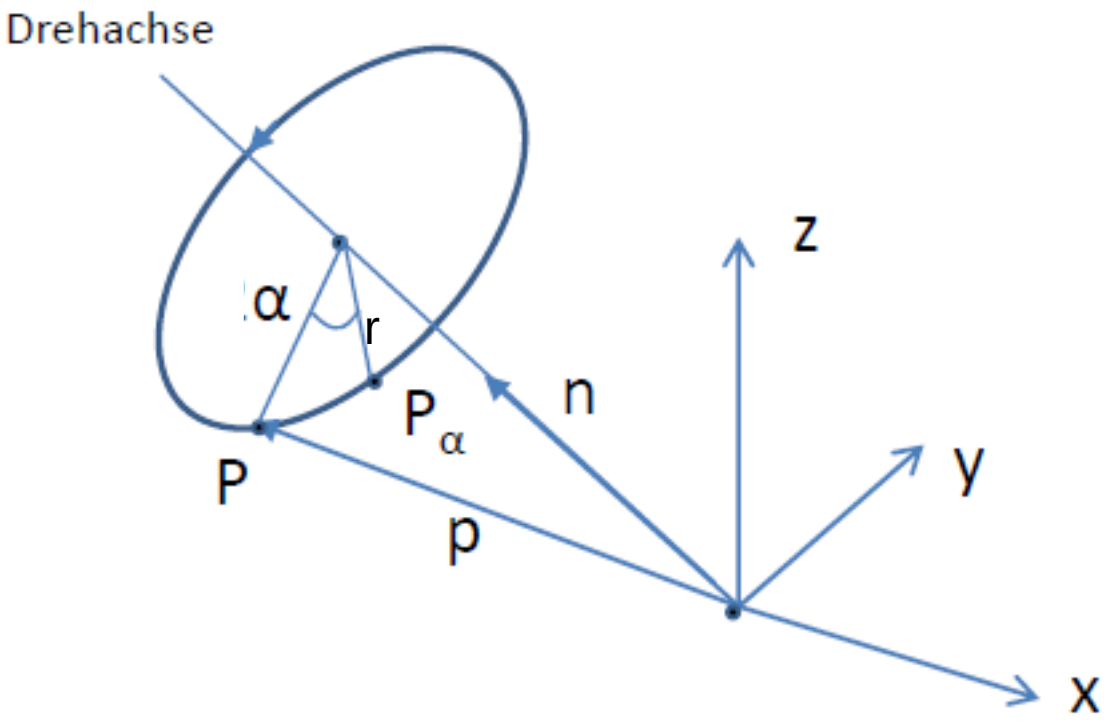
$$q = \underbrace{\cos\frac{\alpha}{2}}_{q_0} + \underbrace{\frac{\vec{a}}{|\vec{a}|}\cdot\sin\frac{\alpha}{2}}_{\vec{q}}$$

$$Q = \cos(\alpha/2) + \vec{a}*\sin(\alpha/2)$$

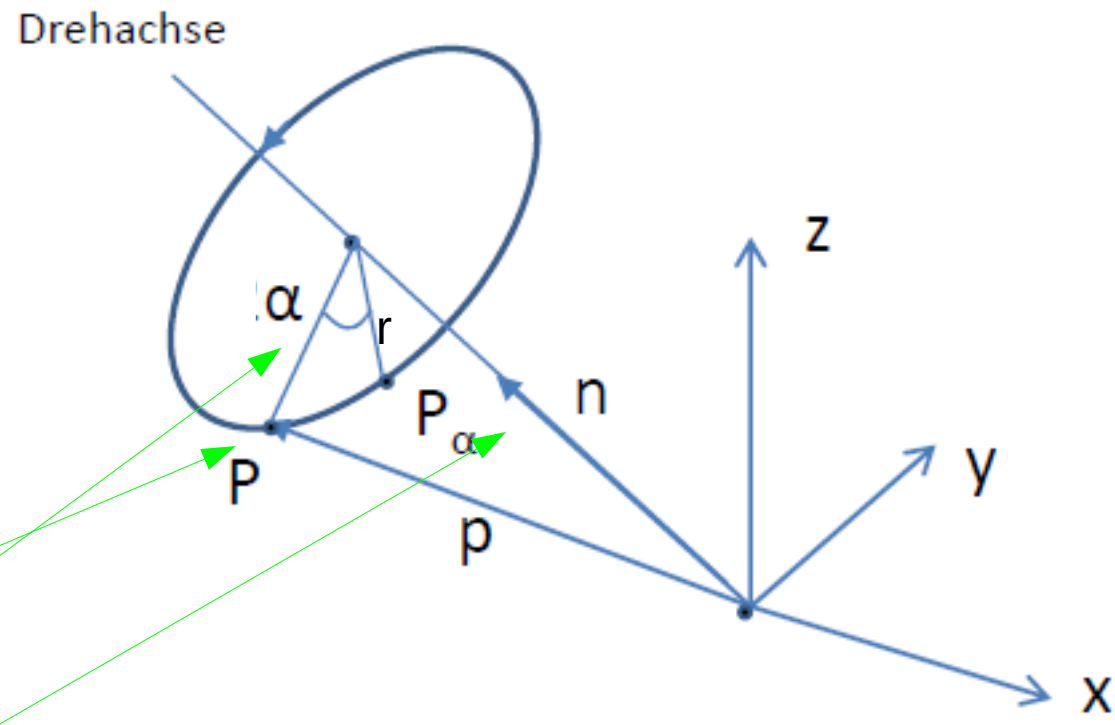
$$Q^* = \cos(-\alpha/2) + \vec{a}*\sin(-\alpha/2) = \cos(\alpha/2) - \vec{a}*\sin(\alpha/2)$$

Well, of course: If I inverse the rotation vector, I rotate in the other direction

Rotation Axis



Construction of Quaternion from Angle & Rotation Axis



P = (0, (x, y, z))

Q = cos(α/2) + $\vec{n}$*sin(α/2) und |n| = 1

n ist ein 3D Vektor

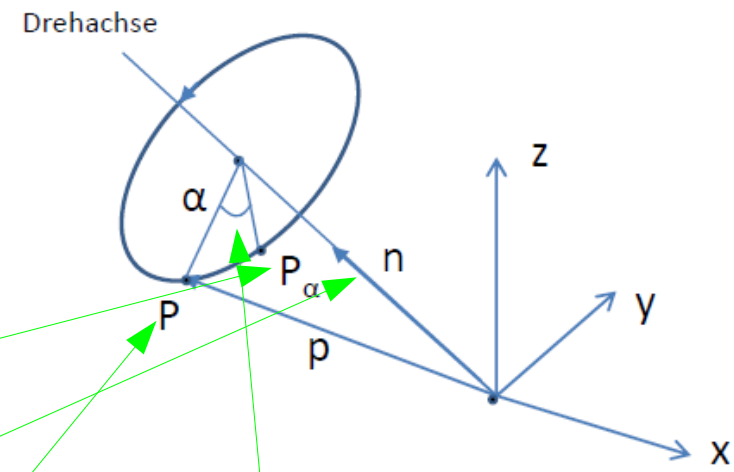
Rotations with Quaternions

$$\underbrace{\vec{w}}_{\mathbb{R}^3} = \underbrace{q}_{\mathbb{R}^4} \odot \underbrace{\vec{v}}_{\mathbb{R}^3} \odot \underbrace{q^*}_{\mathbb{R}^4}$$

ROTATIONS-QUATERNION

COMPLEX-CONJUGIERT

v = (0, x, y, z)
q = cos(α/2) + n*sin(α/2)
w = q * Quaternion(0,v) * invers(q)



$$(0, x', y', z') = (\cos(\alpha/2) + \vec{n}*\sin(\alpha/2)) \odot (0,x,y,z) \odot (\cos(\alpha/2) - \vec{n}*\sin(\alpha/2))$$

$$p' = Q \odot p \odot Q^*$$

Concatenated Rotations

A = Quaternion, encoding the first rotation
B = the second one
C = the third one

Total D = $C \odot B \odot A$

And backwards: $A^* \odot B^* \odot C^*$

at each Q
$|Q| = 1$
otherwise our vectors will be lengethened or shortened
...
but $\odot$ does not change the attitude

rotation of vector v:

$w = Q \odot v \odot Q^*$

And backwards:

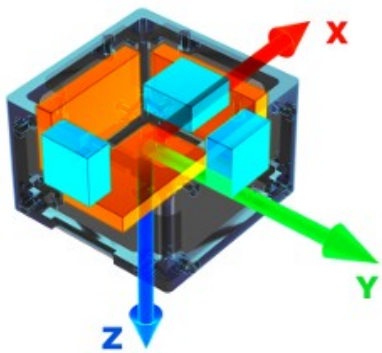
$v = Q^* \odot w \odot Q$

$v = Q^* \odot (Q \odot v \odot Q^*) \odot Q$

$Q^* \odot Q = 1$

Quaternion for 3D Rotations

around 3 corners: first Z then Y then X



Beware: Gyro output is NOT angle,
but rotation rate (3D vector w)

$(\alpha, \beta, \psi) = \Delta T * w$

My attitude (Quaternion), I call it Q (initially (1,(0,0,0))

Rot(X, α) → Quaternion Qx: $(\cos(\alpha/2), \mathbf{(1, 0, 0)}*\sin(\alpha/2)) = (\cos(\alpha/2), (\sin(\alpha/2), 0, 0))$

Rot(Y, β) → Quaternion Qy: $(\cos(\beta/2), \mathbf{(0, 1, 0)}*\sin(\beta/2)) = (\cos(\beta/2), (0, \sin(\beta/2), 0))$

Rot(Z, ψ) → Quaternion Qz: $(\cos(\psi/2), \mathbf{(0, 0, 1)}*\sin(\psi/2)) = (\cos(\psi/2), (0, 0, \sin(\psi/2)))$

Total rotation quaternion: rot = $Qx \odot Qy \odot Qz$

Why this order?
from: RPY
for micro integration of micro rotation

New attitude: $Q = rot \odot Q$

Or the other way round?

Quaternion for 3D Rotationen
The direct path (no xyz corners)

Direct

Beware: Gyro output is NOT angle, but rotation rate (3D vector w)
My attitude (Quaternion), I call it Q

w = 3D vector, with rotation rate
ΔT = period, let‘s try to minimise it -> fairly few arithmetic operations

sin & cos are very time consuming!

a (rotation axis) = normalize(w)
α (angle around a) = $|w| * \Delta T$

rotor: rot = ... see next slide

New attitude: Q = rot $\odot$ Q

Or the other way round?

3 Ways for the Same

Rot(X, α) → Quaternion Qx: (cos(α/2), (1, 0, 0)*sin(α/2)) = (cos(α/2), (sin(α/2), 0, 0))
Rot(Y, β) → Quaternion Qy: (cos(β/2), (0, 1, 0)*sin(β/2)) = (cos(β/2), (0, sin(β/2), 0))
Rot(Z, ψ) → Quaternion Qz: (cos(ψ/2), (0, 0, 1)*sin(ψ/2)) = (cos(ψ/2), (0, 0, sin(ψ/2)))

rot = Qx ⊙ Qy ⊙ Qz
New attitude: Q = rot ⊙ Q

sin & cos are very time consuming!

a (Rotationachse) = normalize(w)
α (Winkel um a) = |w| * ΔT
Drehquaternion: rot = Quaternion(cos(α/2) , a * sin(α/2))
Neune Lage: Q = rot ⊙ Q

sin & cos sind sehr aufwändig!

(aus „a practical method for implementing an attitude and heading reference systen, Rodrigo Munguia & antoni Grau“)
nicht genau, aber eine sehr gute **Approximation** bei langsam (normal) drehen und 3x Schneller)
https://math.stackexchange.com/questions/1693067/differences-between-quaternion-integration-methods

dQ/dt = 1/2 Q ⊙ [0, w] (Nur als info)
rot = (1/2 Q ⊙ [0, w]) * ΔT
Q = normier(Q + rot) // + und nicht ⊙ !! Wie? aber funktioniert!

(Keiner von uns wiess wie, aber es funktioniert!)

Aproximation, aber schneller und dadurch genauer als 1 & 2!!

3 Ways for the Same

Rot(X, α) → Quaternion Qx: (cos(α/2), (1, 0, 0)*sin(α/2)) = (cos(α/2), (sin(α/2), 0, 0))
Rot(Y, β) → Quaternion Qy: (cos(β/2), (0, 1, 0)*sin(β/2)) = (cos(β/2), (0, sin(β/2), 0))
Rot(Z, ψ) → Quaternion Qz: (cos(ψ/2), (0, 0, 1)*sin(ψ/2)) = (cos(ψ/2), (0, 0, sin(ψ/2)))

rot = Qx ⊙ Qy ⊙ Qz
Neune Lage: Q = rot ⊙ Q

sin & cos sind sehr aufwändig!

a (rotation axis) = normalize(w)
α (angle around a) = |w| * ΔT
rotation quaternion: rot = Quaternion(cos(α/2) , a * sin(α/2))
New attitude: Q = rot ⊙ Q

sin & cos are very time consuming!

(aus „a practical method for implementing an attitude and heading reference systen, Rodrigo Munguia & antoni Grau“)
nicht genau, aber eine sehr gute **Approximation** bei langsam (normal) drehen und 3x Schneller)
https://math.stackexchange.com/questions/1693067/differences-between-quaternion-integration-methods

dQ/dt = 1/2 Q ⊙ [0, w] (Nur als info)
rot = (1/2 Q ⊙ [0, w]) * ΔT
Q = normier(Q + rot) // + und nicht ⊙ !! Wie? aber funktioniert!

(Keiner von uns wiess wie, aber es funktioniert!)

Aproximation, aber schneller und dadurch genauer als 1 & 2!!

3 Ways for the Same

Rot(X, α) → Quaternion Qx: (cos(α/2), (1, 0, 0)*sin(α/2)) = (cos(α/2), (sin(α/2), 0, 0))
Rot(Y, β) → Quaternion Qy: (cos(β/2), (0, 1, 0)*sin(β/2)) = (cos(β/2), (0, sin(β/2), 0))
Rot(Z, ψ) → Quaternion Qz: (cos(ψ/2), (0, 0, 1)*sin(ψ/2)) = (cos(ψ/2), (0, 0, sin(ψ/2)))

rot = Qx ⊙ Qy ⊙ Qz
Neune Lage: Q = rot ⊙ Q

sin & cos sind sehr aufwändig!

a (Rotationachse) = normalize(w)
α (Winkel um a) = |w| * ΔT
Drehquaternion: rot = Quaternion(cos(α/2) , a * sin(α/2))
Neune Lage: Q = rot ⊙ Q

sin & cos sind sehr aufwändig!

(from „a practical method for implementing an attitude and heading reference systen, Rodrigo Munguia & Antoni Grau“)
not exact, but a very good **Approximation** when rotating slowly (normal) and 3x faster)

https://math.stackexchange.com/questions/1693067/differences-between-quaternion-integration-methods

dQ/dt = 1/2 Q ⊙ [0, w] (Only as info)
rot = (1/2 Q ⊙ [0, w]) * ΔT
Q = normalised(Q + rot) //+ and not ⊙ !! How? It just works!

(None of us knows how, but it works!)

Approximation, but faster and thus more precise than 1 & 2!!

3 Ways for the Same

Rot(X, α) → Quaternion Qx: (cos(α/2), (1, 0, 0)*sin(α/2)) = (cos(α/2), (sin(α/2), 0, 0))
Rot(Y, β) → Quaternion Qy: (cos(β/2), (0, 1, 0)*sin(β/2)) = (cos(β/2), (0, sin(β/2), 0))
Rot(Z, ψ) → Quaternion Qz: (cos(ψ/2), (0, 0, 1)*sin(ψ/2)) = (cos(ψ/2), (0, 0, sin(ψ/2)))

rot = Qx ⊙ Qy ⊙ Qz
New attitude: Q = rot ⊙ Q

sin & cos are very time consuming!

a (Rotationachse) = normalize(w)
α (Winkel um a) = |w| * ΔT
Drehquaternion: rot = Quaternion(cos(α/2) , a * sin(α/2))
New attitude: Q = rot ⊙ Q

sin & cos are very time consuming!

(aus „a practical method for implementing an attitude and heading reference systen, Rodrigo Munguia & Antoni Grau“)
not exact, but a very good **Approximation** when rotating slowly (normal) and 3x faster)

https://math.stackexchange.com/questions/1693067/differences-between-quaternion-integration-methods

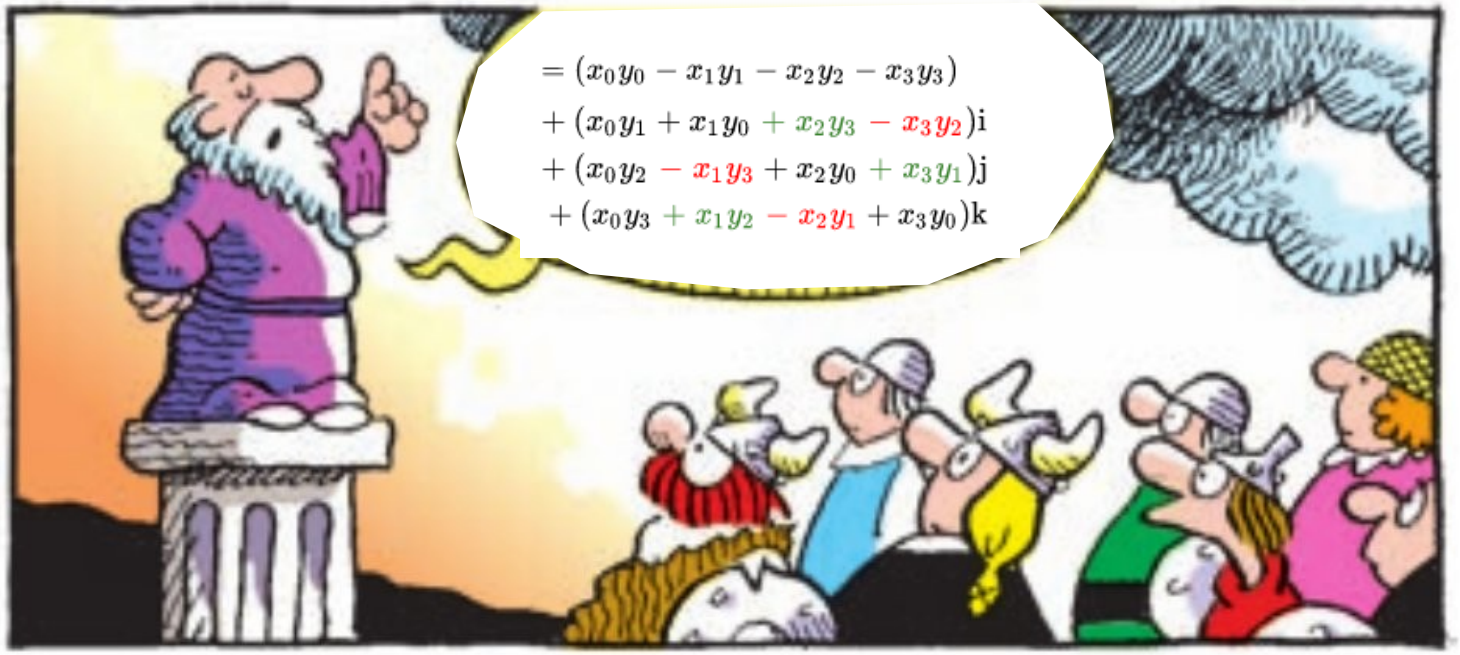
dQ/dt = 1/2 Q ⊙ [0, w] (Only as info)
rot = (1/2 Q ⊙ [0, w]) * ΔT
Q = normalised(Q + rot) //+ and not ⊙ !! How? It just works!

(None of us knows how, but it works!)

Approximation, but faster and thus more precise than 1 & 2!!

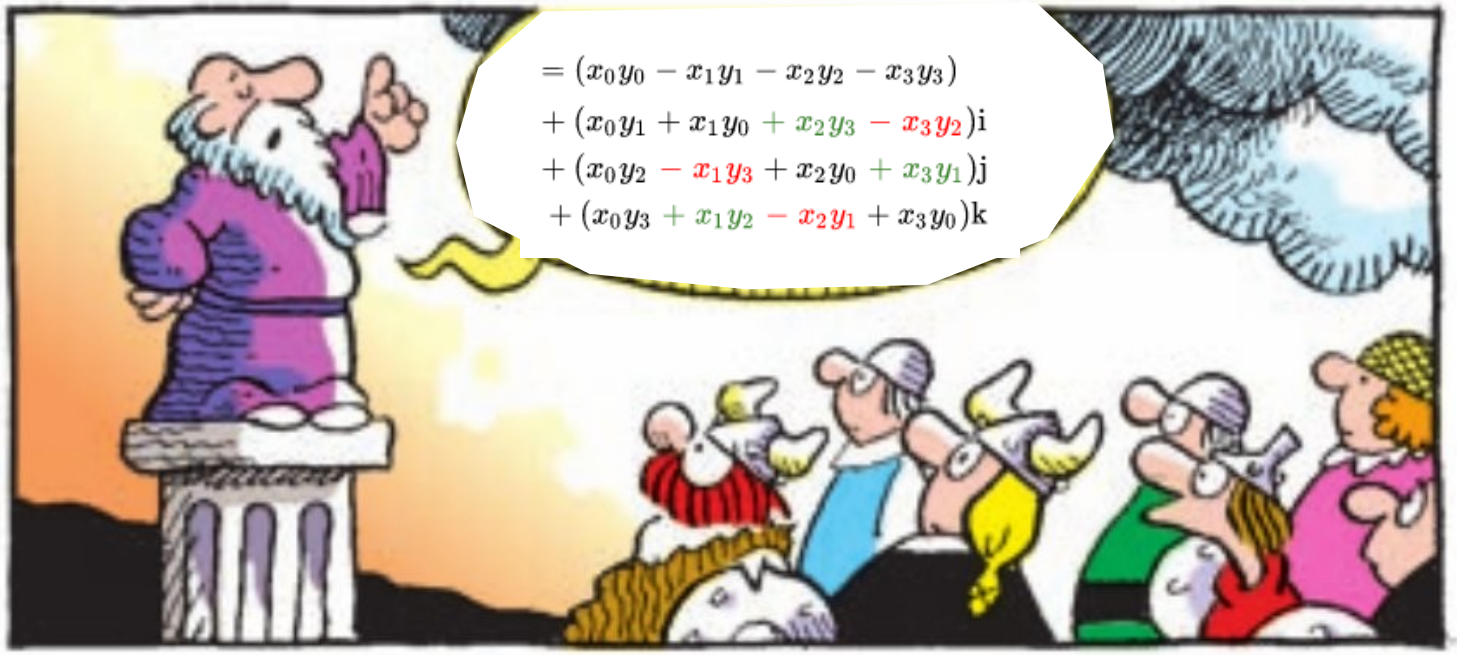
What is it
great wise man?

What is it?!!



What is it great wise man?

What is it?!!



$$= (x_0y_0 - x_1y_1 - x_2y_2 - x_3y_3)$$
$$+ (x_0y_1 + x_1y_0 + x_2y_3 - x_3y_2)\mathrm{i}$$
$$+ (x_0y_2 - x_1y_3 + x_2y_0 + x_3y_1)\mathrm{j}$$
$$+ (x_0y_3 + x_1y_2 - x_2y_1 + x_3y_0)\mathrm{k}$$

Operators for Quaternions

$Q + R = (q_0 + \mathbf{q}) + (r_0 + \mathbf{r}) = (q_0 + r_0 + \mathbf{q} + \mathbf{r})$

$Q^* = (q_0 + -\mathbf{q}) = (q_0 - \mathbf{q})$

$|Q| = \mathrm{sqrt}(q_0^2 + q_1^2 + q_2^2 + q_3^2) = \mathrm{sqrt}(q_0^2 + q_x^2 + q_y^2 + q_z^2)$

Q normalised = Q / |Q| Never (0, (0,0,0))

$Q \odot R = (q_0 + \mathbf{q}) \odot (r_0 + \mathbf{r})$ = as an example: how do you multiply (a+**b**)*(c+**d**)

ac + a**d** +**b**c +**bd**

$\odot$ Clifford Product

looks like a vector, coming towards us :)

$Q \odot R = (q_0 + \mathbf{q}) \odot (r_0 + \mathbf{r})$ = as an example: how do you multiply (a+**b**)*(c+**d**)

ac + a**d** +**b**c +**bd**

Which one is scalar and which one is vector?

And if **b** and **d** are vectors? Scalar- or Vector-Product?

But, if **b and d**) are imaginary values, thenn

ac + a**d** +**b**c -**bd**

$Q = q0 + \mathbf{q}$

$$Q = \left(q0 + \begin{bmatrix} x \\ y \\ z \end{bmatrix} \right)$$

$\odot$ Clifford Product

scalar or vector product?

$$Q \odot R = (q_0 + \mathbf{q}) \odot (r_0 + \mathbf{r}) = (q_0 * r_0 + q_0 * \mathbf{r} + \mathbf{q} * r_0 + \mathbf{q}\ (*?)\ \mathbf{r})$$

$\mathbb{R}4 \odot \mathbb{R}4 \rightarrow \mathbb{R}4$

$$Q \odot R = (q_0 + \mathbf{q}) \odot (r_0 + \mathbf{r}) = \underbrace{\qquad}_{\text{scalar}} \quad \underbrace{\qquad}_{\text{3d vector}}$$

(*?) Both! Scalar- + Vector-Product !!

$$Q = q0 + \mathbf{q}$$

$$Q = \left(q0 + \begin{bmatrix} x \\ y \\ z \end{bmatrix} \right)$$

$\odot$ Clifford Product

$$Q \odot R = (q_0 + \mathbf{q}) \odot (r_0 + \mathbf{r}) = (q_0 * r_0 + q_0 * \vec{\mathbf{r}} + \vec{\mathbf{q}} * r_0 + \vec{\mathbf{q}} \; (*?) \; \vec{\mathbf{r}})$$

$\mathbb{R}4 \odot \mathbb{R}4 \rightarrow \mathbb{R}4$

i * i = -1

$$\boxed{Q \odot R = (q_0 + \mathbf{q}) \odot (r_0 + \mathbf{r}) = (\underbrace{q_0 \; r_0 - \mathbf{q} \bullet \mathbf{r}}_{\text{skalar}} + \underbrace{q_0 \, \mathbf{r} + r_0 \, \mathbf{q} + \mathbf{q} \times \mathbf{r}}_{\text{3d Vektor}})}$$

Why -(q.r)?
Obvious, but why not at qXr?
30 pages of mathematics with (i,j,k)

That's easy to remember, right?

$\odot$ Clifford Product

Original Version / Proof by Sir Hamilton
nothing about scalar and vector, only from real to 3x imaginary

$$Q = x_0 + x_1 i + x_2 j + x_3 k$$
$$R = y_0 + y_1 i + y_2 j + y_3 k$$

$$
\begin{aligned}
Q \odot R \quad &= (x_0 y_0 - x_1 y_1 - x_2 y_2 - x_3 y_3) \\
&+ (x_0 y_1 + x_1 y_0 + x_2 y_3 - x_3 y_2)\mathrm{i} \\
&+ (x_0 y_2 - x_1 y_3 + x_2 y_0 + x_3 y_1)\mathrm{j} \\
&+ (x_0 y_3 + x_1 y_2 - x_2 y_1 + x_3 y_0)\mathrm{k}
\end{aligned}
$$

Who will remember it?

Rotation View

Rotation matrix

Quaternions

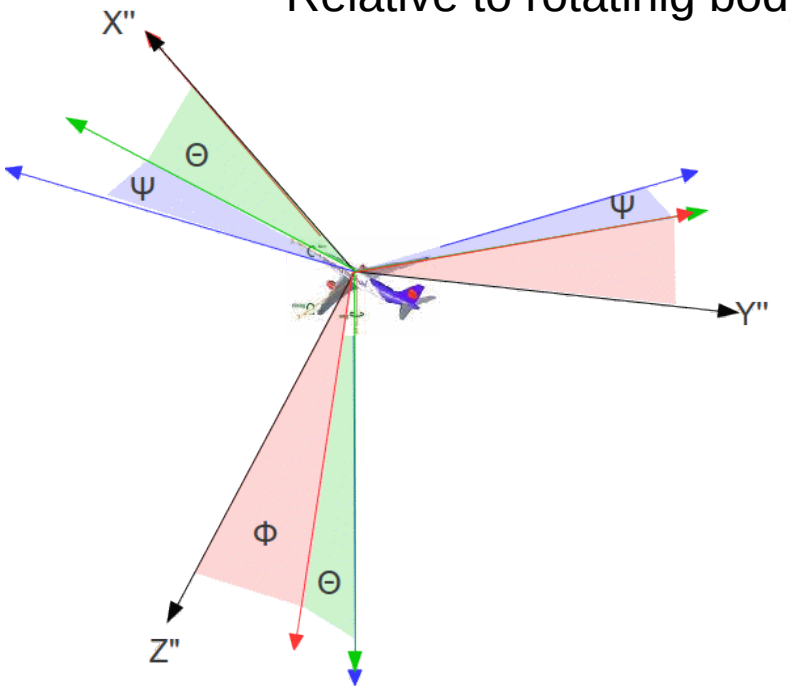
Euler-Angle (incl. RPY)

rotation vector, rotation angle

Euler-Angle

Relative to rotatinig body

12 Variants!



1 rotation in 3 axes
but from each axis
I can observe only 2

3*3*3 (27) possibilities for x,y,z, but each (a,a,b) (a, b,b) and (a,a,a) is useless

12 mögliche Darstellungen:

(X,Y,X)	(X,Z,X)	(Y,X,Y)	(Y,Z,Y)	(Z,X,Y)	(Z,Y,Z)
(X,Y,Z)	(X,Z,Y)	(Y,X,Z)	(Y,Z,X)	(Z,X,Z)	(Z,Y,X)

= Yaw-Pitch-Roll (YPR)
Name: RPY
first Z, (Yaw)
then Y (Pitch)
then X (Roll)
from right to left

first Z (Yaw), then Y(Pitch), then X(Roll) BEWARE of order
to multiply:
XYZ → RPY (Beware: Y-axis and Y of Yaw)

Warning

12 mögliche Darstellungen:

(X,Y,X)	(X,Z,X)	(Y,X,Y)	(Y,Z,Y)	(Z,X,Y)	(Z,Y,Z)
(X,Y,Z)	(X,Z,Y)	(Y,X,Z)	(Y,Z,X)	(Z,X,Z)	(Z,Y,X)

Only valid for 100% sequenced rotations
e.g.
- first X
- then Y
- then X

Not suitable for IMU Measurements!
- in IMU all 3 axes rotate simultaneously
- For IMU I must use all 3: X, Y, Z measurements
- Order? No matter which, but DIN 9300: RPY : 1. Z 2. Y 3. Z

Example

(X, Y, X)

I want to rotate around Z rotieren, how?

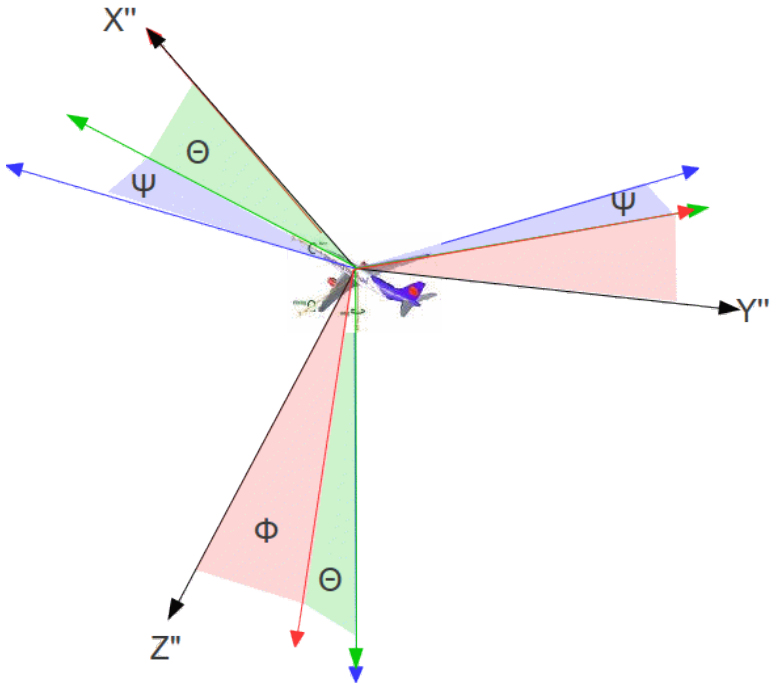
example: my 3d coordinate toy

If I want to rotate from A to B, how shall I do it?

→ too difficult, rather by rotation vector e.g. quaternions.

Euler-Angle (here YPR)

aviation standard (DIN 9300) (Yaw-Pitch-Roll, Z, Y', X'')



Yaw-Angle (yaw) Ψ
Z Axis → X-Y Plane

Pitch-Angle (pitch) Θ,
Y Axis → X-Z Plane

Roll-Angle (roll) Φ,
X Axis → Y-Z Plane

.

Rot(X, α) → Quaternion Qx: (cos(α/2), (1, 0, 0)*sin(α/2)) = (cos(α/2), (sin(α/2), 0, 0))
Rot(Y, β) → Quaternion Qy: (cos(β/2), (0, 1, 0)*sin(β/2)) = (cos(β/2), (0, sin(β/2), 0))
Rot(Z, ψ) → Quaternion Qz: (cos(ψ/2), (0, 0, 1)*sin(ψ/2)) = (cos(ψ/2), (0, 0, sin(ψ/2)))

Qx ⊙ Qy ⊙ Qz

Euler-Angle

for your information only

Z – X – Z'

First you rotate by an angle Ψ around z-Axis of the global coordinate system (Z).
Then follows a rotation by the angle Θ around the new x-Axis (X')
and finally by the angle Φ around the new z-Axis Z'' derived from both rotations before .

$$M_{zxz} = \begin{pmatrix} \cos\Psi & -\sin\Psi & 0 \\ \sin\Psi & \cos\Psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\Theta & -\sin\Theta \\ 0 & \sin\Theta & \cos\Theta \end{pmatrix} \begin{pmatrix} \cos\Phi & -\sin\Phi & 0 \\ \sin\Phi & \cos\Phi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} \cos\Psi\cos\Phi - \sin\Psi\cos\Theta\sin\Phi & -\cos\Psi\sin\Phi - \sin\Psi\cos\Theta\cos\Phi & \sin\Psi\sin\Theta \\ \sin\Psi\cos\Phi + \cos\Psi\cos\Theta\sin\Phi & \cos\Psi\cos\Theta\cos\Phi - \sin\Psi\sin\Phi & -\cos\Psi\sin\Theta \\ \sin\Theta\sin\Phi & \sin\Theta\cos\Phi & \cos\Theta \end{pmatrix}$$

Euler-Angle

for your information only

Etc., etc., etc.

12 mögliche Darstellungen:

(X,Y,X)	(X,Z,X)	(Y,X,Y)	(Y,Z,Y)	(Z,X,Y)	(Z,Y,Z)
(X,Y,Z)	(X,Z,Y)	(Y,X,Z)	(Y,Z,X)	(Z,X,Z)	(Z,Y,X)